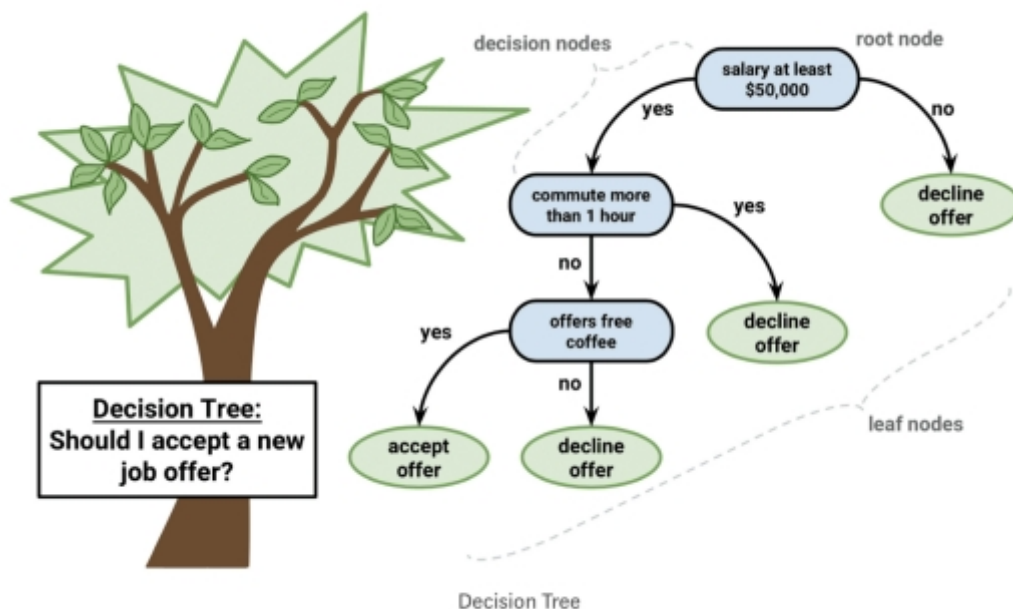


章节1：决策树的特点与数学表达

决策树是属于有监督机器学习的一种，起源非常早，符合直觉并且非常直观，模仿人类做决策的过程，早期人工智能模型中有很多应用，现在更多的是使用基于决策树的一些集成学习的算法。这章我们把决策树算法理解透彻非常有利于后面去学习集成学习。



特点

1. 可以处理非线性的问题
2. 可解释性强 没有 θ
3. 模型简单，模型预测效率高if else
4. 不容易显示的使用函数表达，不可微 XGBoost

决策树模型生成和预测

模型生成：通过大量数据生成一颗非常好的树，用这棵树来预测新来的数据

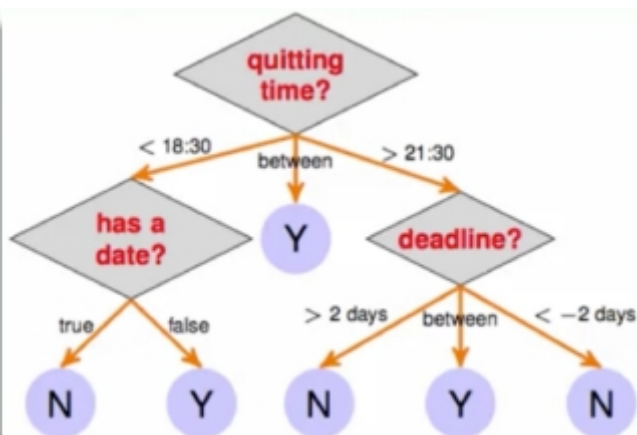
预测：来一条新数据，按照生成好的树的标准，落到某一个节点上

决策树的数学表达形式

A.整体方式表达

$$G(\mathbf{x}) = \sum_{t=1}^T q_t(\mathbf{x}) \cdot g_t(\mathbf{x})$$

- **base hypothesis** $g_t(\mathbf{x})$:
leaf at end of path t ,
a **constant** here
- **condition** $q_t(\mathbf{x})$:
[is $\mathbf{x}$ on path t ?]



B. 迭代方式表达

Recursive View

$$G(\mathbf{x}) = \sum_{c=1}^C \llbracket b(\mathbf{x}) = c \rrbracket \cdot G_c(\mathbf{x})$$

- $G(\mathbf{x})$: full-tree hypothesis
- $b(\mathbf{x})$: branching criteria
- $G_c(\mathbf{x})$: sub-tree hypothesis at the c -th branch



构建决策树数学表达

A Basic Decision Tree Algorithm

$$G(\mathbf{x}) = \sum_{c=1}^C \llbracket b(\mathbf{x}) = c \rrbracket G_c(\mathbf{x})$$

function **DecisionTree**(data $\mathcal{D} = \{(\mathbf{x}_n, y_n)\}_{n=1}^N$)

if **termination criteria met**

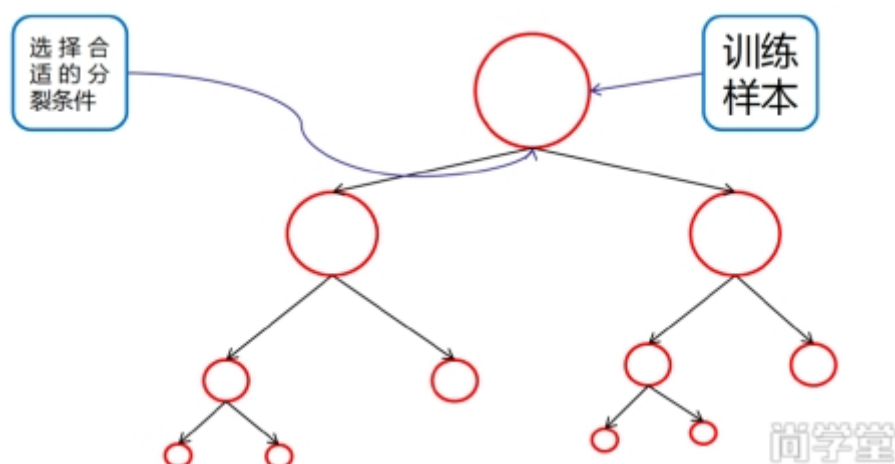
return **base hypothesis** $g_t(\mathbf{x})$

else

- 1 learn **branching criteria** $b(\mathbf{x})$
- 2 split $\mathcal{D}$ to C parts $\mathcal{D}_c = \{(\mathbf{x}_n, y_n) : b(\mathbf{x}_n) = c\}$
- 3 build sub-tree $G_c \leftarrow \text{DecisionTree}(\mathcal{D}_c)$
- 4 return $G(\mathbf{x}) = \sum_{c=1}^C \llbracket b(\mathbf{x}) = c \rrbracket G_c(\mathbf{x})$

four choices: **number of branches**, **branching criteria**, **termination criteria**, & **base hypothesis**

总结决策树流程和待解决问题？



1.将原始数据集进行筛选，分裂成子数据集

每次分成几份？

以什么条件进行划分？

2.对生成的子数据集不断分裂，直到停止

停止的条件是什么？

3.利用最终生成的n份数据的共性来代表这个节点

如何用节点共性代表未来预测值？

总结，决策树的生成说白了就是数据不断分裂的递归过程，每一次分裂，尽可能让类别一样的数据在树的一边，当树的叶子节点的数据都是一类的时候，则停止分裂。

数据集



分割方式1



分割方式2



章节2：生成决策树所需分裂指标

常用分裂条件

- Gini系数
 - 信息增益
 - 信息增益率
- 纯度
- MSE (回归问题)

Gini系数 (CART)

基尼系数是指国际上通用的、用以衡量一个国家或地区居民收入差距的常用指标。

基尼系数最大为“1”，最小等于“0”。基尼系数越接近0表明收入分配越是趋向平等。国际惯例把0.2以下视为收入绝对平均，0.2-0.3视为收入比较平均；0.3-0.4视为收入相对合理；0.4-0.5视为收入差距较大，当基尼系数达到0.5以上时，则表示收入悬殊。

基尼指数最早由意大利统计与社会学家Corrado Gini在1912年提出。

其具体含义是指，在全部居民收入中，用于进行不平均分配的那部分收入所占的比例。基尼系数最大为“1”，最小等于“0”。前者表示居民之间的收入分配绝对不平均，即100%的收入被一个单位的人全部占有了；而后者则表示居民之间的收入分配绝对平均，即人与人之间收入完全平等，没有任何差异。但这两种情况只是在理论上的绝对化形式，在实际生活中一般不会出现。因此，基尼系数的实际数值只能介于0~1之间，基尼系数越小收入分配越平均，基尼系数越大收入分配越不平均。国际上通常把0.4作为贫富差距的警戒线，大于这一数值容易出现社会动荡。

联合国开发计划署等组织规定：

- 若低于0.2表示指数等级极低；（高度平均）
- 0.2-0.29表示指数等级低；（比较平均）
- 0.3-0.39表示指数等级中；（相对合理）
- 0.4-0.59表示指数等级高；（差距较大）
- 0.6以上表示指数等级极高。（差距悬殊）

基尼指数通常把0.4作为收入分配差距的“警戒线”，根据黄金分割律，其准确值应为0.382。一般发达国家的基尼指数在0.24到0.36之间，美国偏高，为0.52。中国国家统计局公布基尼系数2012年为0.474，2013年为0.473，2014年为0.469，2015年为0.462，2016年为0.465。

公式

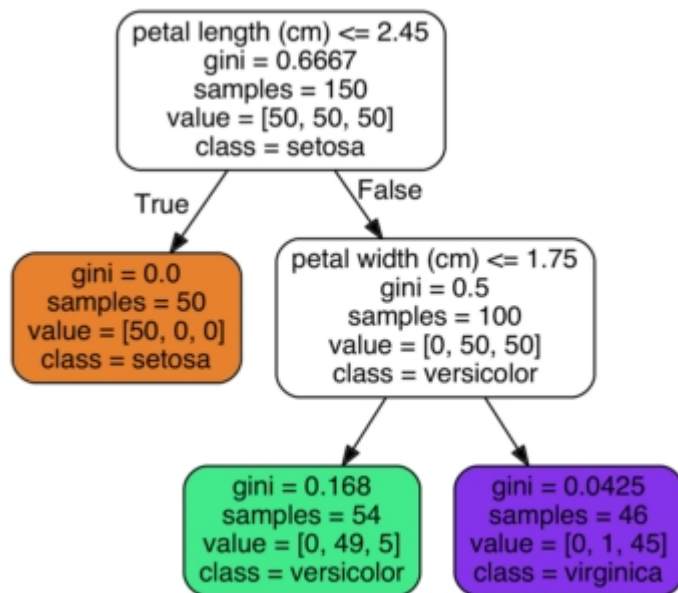
$$Gini(p) = \sum_{k=1}^K p_k(1 - p_k) = 1 - \sum_{k=1}^K p_k^2$$

Gini系数越小，代表D集合中的数据越纯，所有我们可以计算分裂前的值

在按照某个维度对数据集进行划分，然后可以去计算多个节点的Gini系数：

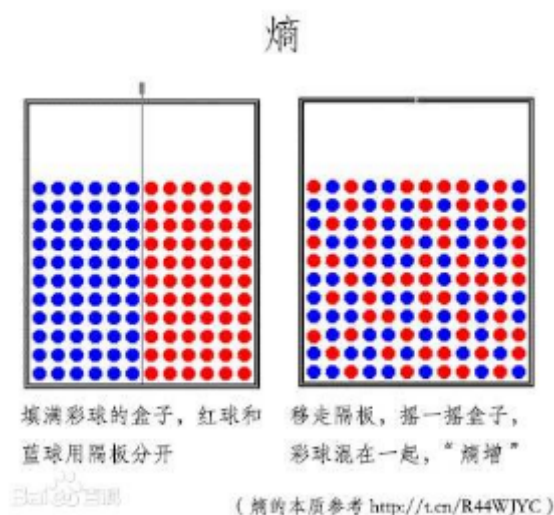
$$Gini(D, A) = \frac{|D_1|}{|D|} Gini(D_1) + \frac{|D_2|}{|D|} Gini(D_2)$$

尝遍百草，哪根草最甜，就选哪根！



信息增益 (ID3)

在信息论里熵叫作信息量，即熵是对不确定性的度量。从控制论的角度来看，应叫不确定性。信息论的创始人香农在其著作《通信的数学理论》中提出了建立在概率统计模型上的信息度量。他把信息定义为“用来消除不确定性的东西”。在信息世界，熵越高，则能传输越多的信息，熵越低，则意味着传输的信息越少。还是举例说明，假设Kathy在买衣服的时候有颜色，尺寸，款式以及设计年份四种要求，而North只有颜色和尺寸的要求，那么在购买衣服这个层面上Kathy由于选择更多因而不确定性因素更大，最终Kathy所获取的信息更多，也就是熵更大。所以信息量=熵=不确定性，通俗易懂。在叙述决策树时我们用熵表示不纯度 (Impurity)。



$$H(N) = -\sum_j P(\omega_j) \log_2 P(\omega_j)$$

信息增益: 分裂前的信息熵 减去 分裂后的信息熵

一个分裂导致的信息增益越大，代表这次分裂提升的纯度越高

uin	gender	act_info	is_lost
1	男	高	0
2	女	中	0
3	男	低	1
4	女	高	0
5	男	高	0
6	男	中	0
7	男	中	1
8	女	中	0
9	女	低	1
10	女	中	0
11	女	高	0
12	男	低	1
13	女	低	1
14	男	高	0
15	男	高	0

	positive	negative	汇总
整体	5	10	15
男性	3	5	8
女性	2	5	7
高	0	6	6
中	1	4	5
低	4	0	4

整体熵：

$$E(S) = -\frac{5}{15} \log_2\left(\frac{5}{15}\right) - \frac{10}{15} \log_2\left(\frac{10}{15}\right) = 0.9182$$

性别熵：

$$E(g_1) = -\frac{3}{8} \log_2\left(\frac{3}{8}\right) - \frac{5}{8} \log_2\left(\frac{5}{8}\right) = 0.9543$$

$$E(g_2) = -\frac{2}{7} \log_2\left(\frac{2}{7}\right) - \frac{5}{7} \log_2\left(\frac{5}{7}\right) = 0.8631$$

性别信息增益：

$$IGain(S, g) = E(S) - \frac{8}{15} E(g_1) - \frac{7}{15} E(g_2) = 0.0064$$

同理计算活跃度熵：

$$E(a_1) = 0 \quad E(a_2) = 0.7219 \quad E(a_3) = 0.0$$

活跃度信息增益：

$$IGain(S, a) = E(S) - \frac{6}{15} E(a_1) - \frac{5}{15} E(a_2) - \frac{4}{15} E(a_3) = 0.6776$$

信息熵与Gini指数关系

首先看二者的定义：

$$\begin{cases} H(X) = -\sum_{k=1}^K p_k \ln p_k \\ \text{Gini}(X) = \sum_{k=1}^K p_k (1 - p_k) \end{cases}$$

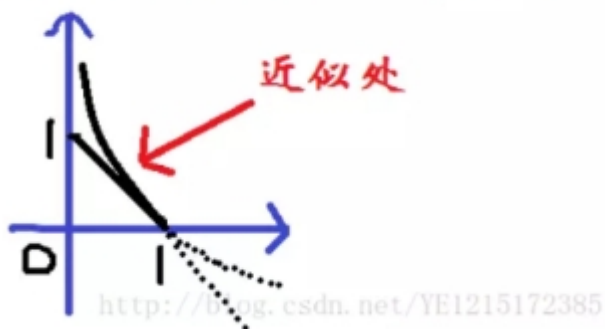
将 $f(x) = -\ln x$ 在 $x = 1$ 处进行一阶泰勒展开（忽略高阶无穷小）：

$$\begin{aligned} f(x) &= f(x_0) + f'(x_0)(x - x_0) + o(\cdot) \\ &= f(1) + f'(1)(x - 1) + o(\cdot) \\ &= 1 - x \end{aligned}$$

因此，熵可近似转化为：

$$\begin{aligned} H(X) &= - \sum_{k=1}^K p_k \ln p_k = \sum_{k=1}^K p_k (-\ln p_k) \\ &\simeq \sum_{k=1}^K p_k (1 - p_k) = \text{Gini}(X) \end{aligned}$$

即用 $-\log P$ 来近似 $1-P$ ，且 P 处于 $[0,1]$ ，从图上来直观理解一下：



基尼指数是信息熵中 $-\log P$ 在 $P = 1$ 处一阶泰勒展开后的结果！所以两者都可以用来度量数据集的纯度，用于描述决策树节点的纯度！

信息增益率 (C4.5)

对于多叉树，如果不限制分裂多少支，一次分裂就可以将信息熵降为0，比如ID3

如何平衡分裂情况与信息增益？

信息增益率：信息增益 除以 类别 本身的熵

uin	gender	act_info	is_lost
1	男	高	0
2	女	中	0
3	男	低	1
4	女	高	0
5	男	高	0
6	男	中	0
7	男	中	1
8	女	中	0
9	女	低	1
10	女	中	0
11	女	高	0
12	男	低	1
13	女	低	1
14	男	高	0
15	男	高	0

	positive	negative	汇总
整体	5	10	15
男性	3	5	8
女性	2	5	7
高	0	6	6
中	1	4	5
低	4	0	4

- 三种分类方法

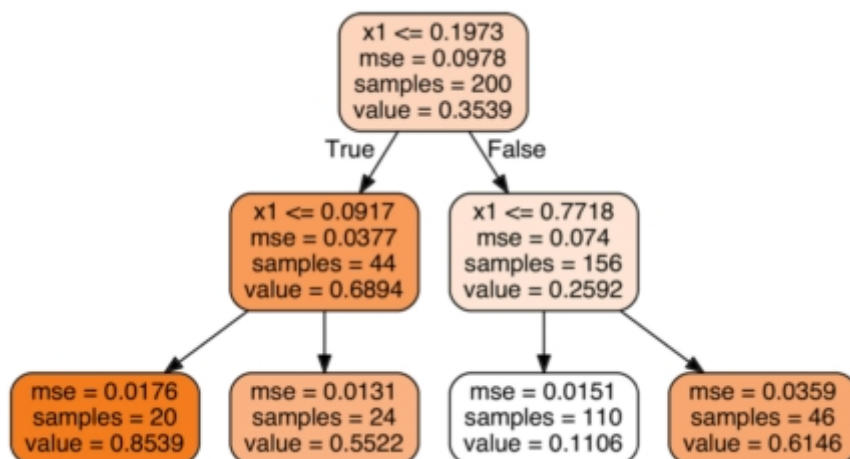
- (H) (M) (L)
- (H,M), (L)
- (H),(M,L)

- 分别计算三种分类方法的信息增益率:

- $GR(1) = \text{Gain} / -(6/15 \cdot \log 6/15 + 5/15 \cdot \log 5/15 + 4/15 \cdot \log 4/15)$
- $GR(2) = \text{Gain} / -(11/15 \cdot \log 11/15 + 4/15 \cdot \log 4/15)$
- $GR(3) = \text{Gain} / -(6/15 \cdot \log 6/15 + 9/15 \cdot \log 9/15)$

MSE

用于回归树



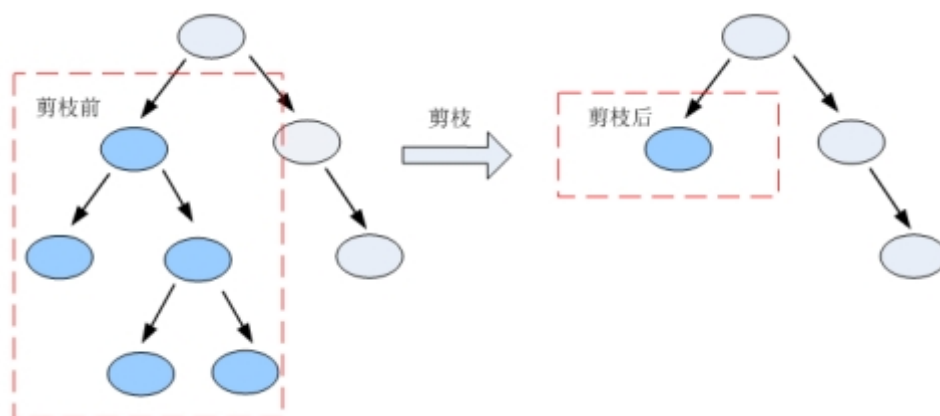
章节3：经典决策树算法

何时停止分裂

什么都不做 (do nothing)

只要无限分下去，最终都会将结果分到最好，fully grown tree，但是容易过拟合

剪枝 (pruning)



是解决过拟合的一个有效方法。当树训练得过于茂盛的时候会出现在测试集上的效果比训练集上差不少的现象，即过拟合。可以采用如下两种剪枝策略

前剪枝

在构建决策树的过程时，提前停止。那么，会将切分节点的条件设置的很苛刻，导致决策树很短小。

```
def __init__(self,
               criterion="gini",
               splitter="best",
               max_depth=None,
               min_samples_split=2,
               min_samples_leaf=1,
               min_weight_fraction_leaf=0.,
               max_features=None,
               random_state=None,
               max_leaf_nodes=None,
               min_impurity_decrease=0.,
               min_impurity_split=None,
               class_weight=None,
               presort=False):
```

后剪枝

决策树构建好后，然后才开始裁剪。

常见三种后剪枝方法：

1. REP—错误率降低剪枝
2. PEP—悲观剪枝（C4.5）
3. CCP—代价复杂度剪枝（CART）

代价复杂度剪枝：

该算法为子树 T_t 定义了代价（cost）和复杂度（complexity），以及一个可由用户设置的衡量代价与复杂度之间关系的参数 α ，其中，代价指在剪枝过程中因子树 T_t 被叶节点替代而增加的错分样本，复杂度表示剪枝后子树 T_t 减少的叶结点数， α 则表示剪枝后树的复杂度降低程度与代价间的关系，定义为：

$$\alpha = \frac{R(t) - R(T_t)}{|N_1| - 1}$$

其中，

$|N_1|$ ：子树 T_t 中的叶节点数；

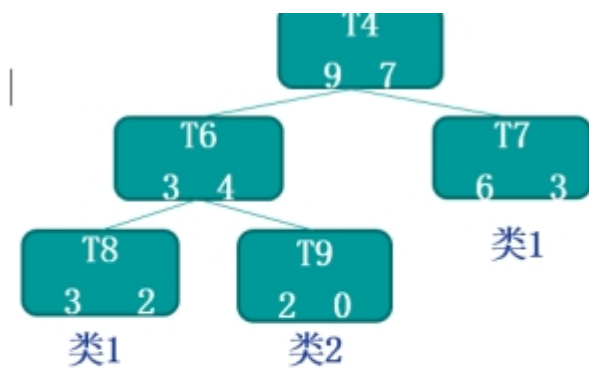
$R(t)$ ：结点 t 的错误代价，计算公式为 $R(t) = r(t) * p(t)$ ，

$r(t)$ 为结点 t 的错分样本率， $p(t)$ 为落入结点 t 的样本占有所有样本的比例；

$R(T_t)$ ：子树 T_t 错误代价，计算公式为 $R(T_t) = \sum R(i)$ ， i 为子树 T_t 的叶节点。

CCP剪枝算法分为两个步骤：

1. 对于完全决策树 T 的每个非叶结点计算 α 值，循环剪掉具有最小 α 值的子树，直到剩下根节点。在该步可得到一系列的剪枝树 $\{T_0, T_1, T_2, \dots, T_m\}$ ，其中 T_0 为原有的完全决策树， T_m 为根结点， T_{i+1} 为对 T_i 进行剪枝的结果；
2. 从子树序列中，根据真实的误差估计选择最佳决策树。



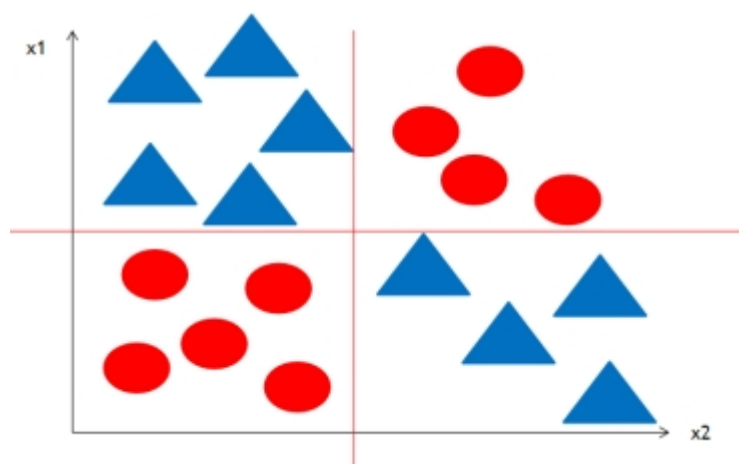
我们以非叶结点T4为例，假设已有的数据有60条，那么

$$R(t) = r(t) * p(t) = (7/16) * (16/60) = 7/60$$

$$R(T_t) = \sum R(i) = (2/5) * (5/60) + (0/2) * (2/60) + (3/9) * (9/60) = 5/60$$

$$\alpha = (R(t) - R(T_t)) / (|N_1| - 1) = 1/60$$

决策树优缺点



优点

1. 决策过程接近人的思维习惯。
2. 模型容易解释，比线性模型具有更好的解释性。
3. 能清楚地使用图形化描述模型。
4. 处理定型特征比较容易。

缺点

1. 一般来说，决策树学习方法的准确率不如其他的模型。针对这种情况存在一些解决方案，在后面的文章中为大家讲解。
2. 不支持在线学习。当有新样本来的时候，需要重建决策树。
3. 容易产生过拟合现象。

经典决策树算法

ID3和C4.5比较

ID3 (Iterative Dichotomiser 3, 迭代二叉树3代) 由Ross Quinlan于1986年提出。1993年, 他对ID3进行改进设计出了C4.5算法。

我们已经知道ID3与C4.5的不同之处在于, ID3根据信息增益选取特征构造决策树, 而C4.5则是以信息增益率为核心构造决策树。既然C4.5是在ID3的基础上改进得到的, 那么这两者的优缺点分别是什么?

使用信息增益会让ID3算法更偏向于选择值多的属性。信息增益反映给定一个条件后不确定性减少的程度, 必然是分得越细的数据集确定性更高, 也就是信息熵越小, 信息增益越大。因此, 在一定条件下, 值多的属性具有更大的信息增益。而C4.5则使用信息增益率选择属性。信息增益率通过引入一个被称作分裂信息(Split information)的项来惩罚取值较多的属性, 分裂信息用来衡量属性分裂数据的广度和均匀性。这样就改进了ID3偏向选择值多属性的缺点。

相对于ID3只能处理离散数据, C4.5还能对连续属性进行处理, 具体步骤为:

1. 把需要处理的样本(对应根节点)或样本子集(对应子树)按照连续变量的大小从小到大进行排序。
2. 假设该属性对应的不同的属性值一共有N个, 那么总共有N-1个可能的候选分割阈值点, 每个候选的分割阈值点的值为上述排序后的属性值中两两前后连续元素的中点, 根据这个分割点把原来连续的属性分成bool属性。实际上可以不用检查所有N-1个分割点。(连续属性值比较多时, 由于需要排序和扫描, 会使C4.5的性能有所下降。)
3. 用信息增益比率选择最佳划分。

C4.5其他优点

1. 在树的构造过程中可以进行剪枝, 缓解过拟合;
2. 能够对连续属性进行离散化处理 (二分法);
3. 能够对缺失值进行处理;

缺点

构造树的过程需要对数据集进行多次顺序扫描和排序, 导致算法低效;

刚才我们提到 信息增益对可取值数目较多的属性有所偏好; 而信息增益率对可取值数目较少的属性有所偏好! OK, 两者结合一下就好了!

解决方法: 先从候选属性中找出信息增益高于平均水平的属性, 再从中选择增益率最高的。而不是大家常说的直接选择信息增益率最高的属性!

CART

CART (Classification And Regression Tree, 分类回归树) 由L.Breiman, J.Friedman, R.Olshen和C.Stone于1984年提出, 是一种应用相当广泛的决策树学习方法。值得一提的是, CART和C4.5一同被评为数据挖掘领域十大算法。

CART算法采用一种二分递归分割的技术, 将当前的样本集分为两个子样本集, 使得生成的每个非叶子节点都有两个分支。因此, CART算法生成的决策树是结构简洁的二叉树。

作为一种决策树学习算法, CART与ID3以及C4.5不同, 它使用基尼系数 (Gini coefficient) 对属性进行选择。我们曾提及基尼系数的计算公式:

$$Gini(p) = \sum_{k=1}^K p_k(1 - p_k) = 1 - \sum_{k=1}^K p_k^2$$

举个例子, 假设我们拥有如下表所示的动物分类信息表:

名称	体温	表面覆盖	胎生	产蛋	能飞	水生	有腿	冬眠	类标记
人	恒温	毛发	是	否	否	否	是	否	哺乳类
巨蟒	冷血	鳞片	否	是	否	否	否	是	爬行类
鲑鱼	冷血	鳞片	否	是	否	是	否	否	鱼类
鲸	恒温	毛发	是	否	否	是	否	否	哺乳类
蛙	冷血	无	否	是	否	有时	是	是	两栖类
巨蜥	冷血	鳞片	否	是	否	否	是	否	爬行类
蝙蝠	恒温	毛发	是	否	是	否	是	否	哺乳类
猫	恒温	皮	是	否	否	否	是	否	哺乳类
豹纹鲨	冷血	鳞片	是	否	否	是	否	否	鱼类
海龟	冷血	鳞片	否	是	否	有时	是	否	爬行类
豪猪	恒温	刚毛	是	否	否	否	是	是	哺乳类
鳗	冷血	鳞片	否	是	否	是	否	否	鱼类
蟾蜍	冷血	无	否	是	否	有时	是	是	两栖类

那么对于非恒温动物，包含爬行类3个、鱼类3个、两栖类2个，因此，

$$Gini1=1-[(3/8) ^2 + (3/8) ^2 + (2/8) ^2]$$

而对于恒温动物，只包含5个哺乳类，同样，

$$Gini2=1 - 1^2 = 0$$

同熵一样，如果样本集合D被某个属性A是否取某个值分成两个样本集合D1和D2，则在属性A的条件下，集合D的基尼指数定义为：

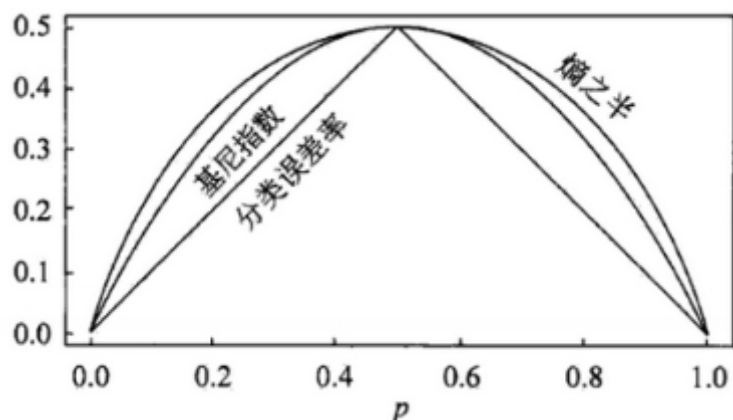
$$Gini(D, A) = \frac{|D_1|}{|D|}Gini(D_1) + \frac{|D_2|}{|D|}Gini(D_2)$$

对于上述按恒温和非恒温属性划分动物，

$$Gini(D, A)= 8/13 * Gini1 + 5/13 * Gini2$$

在一定程度上基尼指数（Gini(D)）反应的是集合D的不确定程度，跟熵类似。Gini(D, A)反应的是经过特征A划分后集合D的不确定程度。基尼系数越大，样本集合的不确定性也就越大。因此最好的属性划分是使得Gini(D, A)最小的划分。

下图显示了二分类问题中基尼系数、熵（单位比特）之半 $1/2 * H(p)$ 和分类误差率的关系。横坐标表示概率p，纵坐标表示损失。可以看出基尼系数和熵之半曲线很接近，都可以近似的代表分类误差率。



每一次选择使基尼系数最小的属性作为切分点直到满足停止条件为止。算法的停止条件是节点中的样本个数小于预定阈值，或者样本集的基尼系数小于预定阈值（样本基本属于同一类），或者没有更多特征。

回归树

除了分类决策之外，CART也可以进行回归决策。

假设X和Y分别为输入和输出变量，Y为连续变量，训练数据集D为：

$$D = \{(\mathbf{X}^1, y^1), (\mathbf{X}^2, y^2), \dots, (\mathbf{X}^n, y^n)\}$$

一个回归树对应着输入空间的一个划分以及在划分的单元上的输出值。假设已经将输入空间划分为M个单元 $R_1, R_2, \dots, R_M$ ，在每个单元 R_m 上有个固定的输出 C_m ，则回归树表示为：

$$f(\mathbf{X}) = \sum_{m=1}^M c_m I(\mathbf{X} \in R_m)$$

问题是怎么对输入空间进行划分。一般采用启发式的思路，选择第j个Feature X_j 和他的取值s分别作为切分变量（splitting variable）和切分点（splitting point），并定义两个区域：

$$R_1(j, s) = \{\mathbf{X} | \mathbf{X}_j \leq s\}, \quad R_2(j, s) = \{\mathbf{X} | \mathbf{X}_j > s\}$$

然后采用平方误差损失求解最优的切分变量j和切分点s。具体地，求解

$$\min_{j,s} [\min_{c_1} \sum_{\mathbf{X}^i \in R_1(j,s)} (y^i - c_1)^2 + \min_{c_2} \sum_{\mathbf{X}^i \in R_2(j,s)} (y^i - c_2)^2]$$

每一个切分变量和切分点对(j,s)都将输入空间分成两个区域，然后分别求每个区域的输出值，使得误差最小，很显然输出值应该是那个区域所有样本值的平均值，即：

$$c_1 = \frac{1}{N_1} \sum_{\mathbf{X}^i \in R_1} y^i, \quad c_2 = \frac{1}{N_2} \sum_{\mathbf{X}^i \in R_2} y^i$$

举例说明，下面有一个简单的训练数据，根据这个数据集我们生成一棵回归树。

x	1	2	3	4	5	6	7	8	9	10
y	5.56	5.70	5.91	6.40	6.80	7.05	8.90	8.70	9.00	4

由于x只有一个Feature，我们不用选择j，下面我们考虑如下的切分点s：1.5, 2.5, 3.5, 4.5, 5.5, 6.5, 7.5, 8.5, 9.5 然后求出对应的 R_1, R_2, c_1, c_2 ，以及总的误差：

$$m(s) = \min_{c_1} \sum_{x^i \in R_1} (y^i - c_1)^2 + \min_{c_2} \sum_{x^i \in R_2} (y^i - c_2)^2$$

经过计算，可以得到如下结果：

s	1.5	2.5	3.5	4.5	5.5	6.5	7.5	8.5	9.5
m(s)	15.72	12.07	8.36	4.78	3.91	1.93	8.01	11.73	15.74

很显然应该取s=6.5作为切分点，此时：

$R_1 = \{1, 2, 3, 4, 5, 6\}, R_2 = \{7, 8, 9, 10\}, c_1 = 6.24, c_2 = 8.91$

决策树为：

$$f(x) = \begin{cases} 6.24, & x < 6.5 \\ 8.91, & x \geq 6.5 \end{cases}$$

然后每个(j,s)对里找出使总误差最小的对作为最终的切分变量和切分点，对切分后的子区域重复这一步骤，直到满足停止条件为止。这样就生成了一颗回归树。此时的回归树成为最小二乘回归树（least squares regression tree）。

总结

C4.5算法是在ID3算法的基础上采用信息增益率的方法选择决策属性。C4.5改进了ID3偏向选择值多属性以及只能处理离散属性等缺点。ID3算法和C4.5算法虽然在对训练样本集的学习中能尽可能多地挖掘信息，但其生成的决策树分支较大，规模较大。为了简化决策树的规模，提高生成决策树的效率，又出现了根据GINI系数来选择测试属性的决策树算法CART。

CART算法采用一种二分递归分割的技术，与基于信息熵的算法不同，CART算法对每次样本集的划分计算GINI系数，GINI系数越小则划分越合理。CART算法总是将当前样本集分割为两个子样本集，使得生成的决策树的每个非叶结点都只有两个分枝。因此CART算法生成的决策树是结构简洁的二叉树。

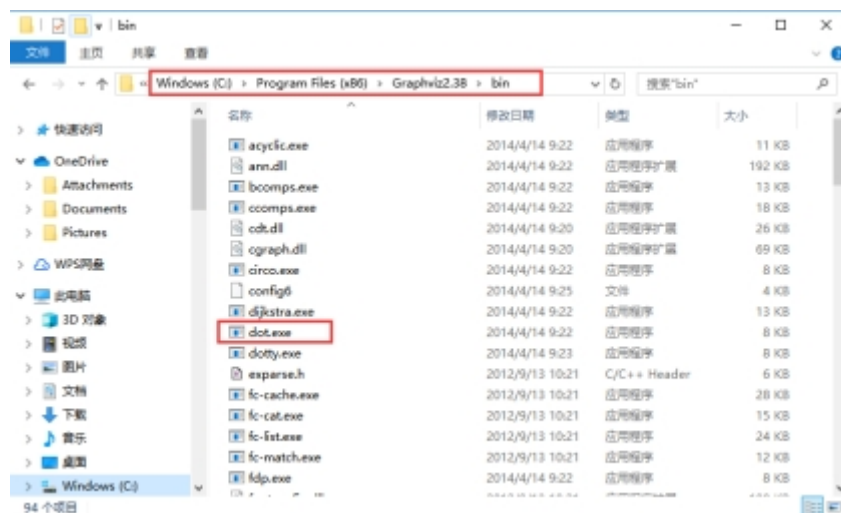
章节4：代码实战对鸢尾花数据集分类

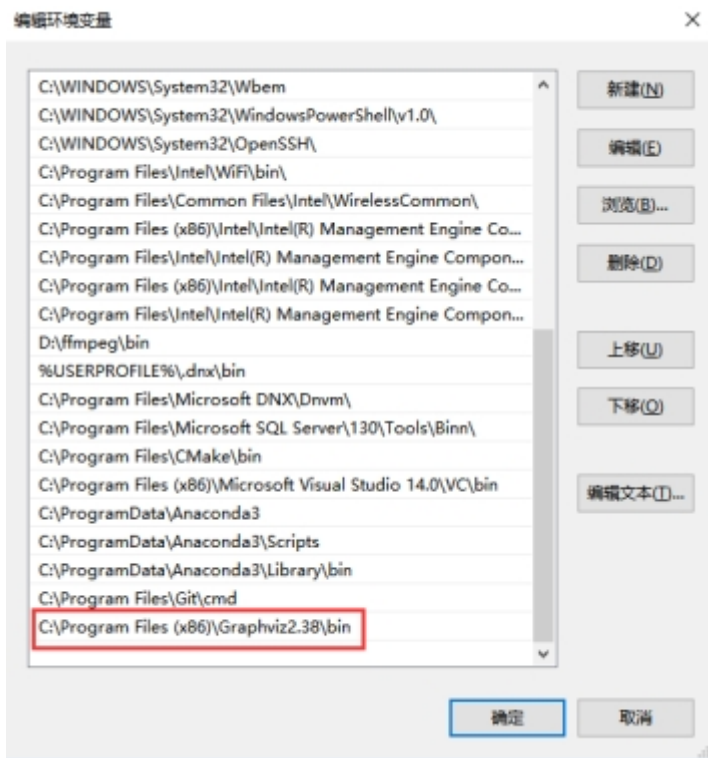
代码实战

使用sklearn库中的DecisionTreeClassifier()方法。这个方法是基于CART的，但是可以使用Information Gain的方法来分，也就是使用C4.5的方法来进行划分，但是依旧是二叉树而不能生成多叉树。

https://graphviz.gitlab.io/_pages/Download/Download_windows.html

下载安装，并配置好环境变量





决策树分类鸢尾花数据集

```
import pandas as pd
import numpy as np
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import export_graphviz
from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
import matplotlib as mpl
```

读取数据

```
iris = load_iris()
data = pd.DataFrame(iris.data)
data.columns = iris.feature_names
data['species'] = load_iris().target
print(data)
```

切分数据

```
x = data.iloc[:, 2:4] # 花萼长度和宽度
y = data.iloc[:, -1]
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.75,
random_state=42)
```

训练模型

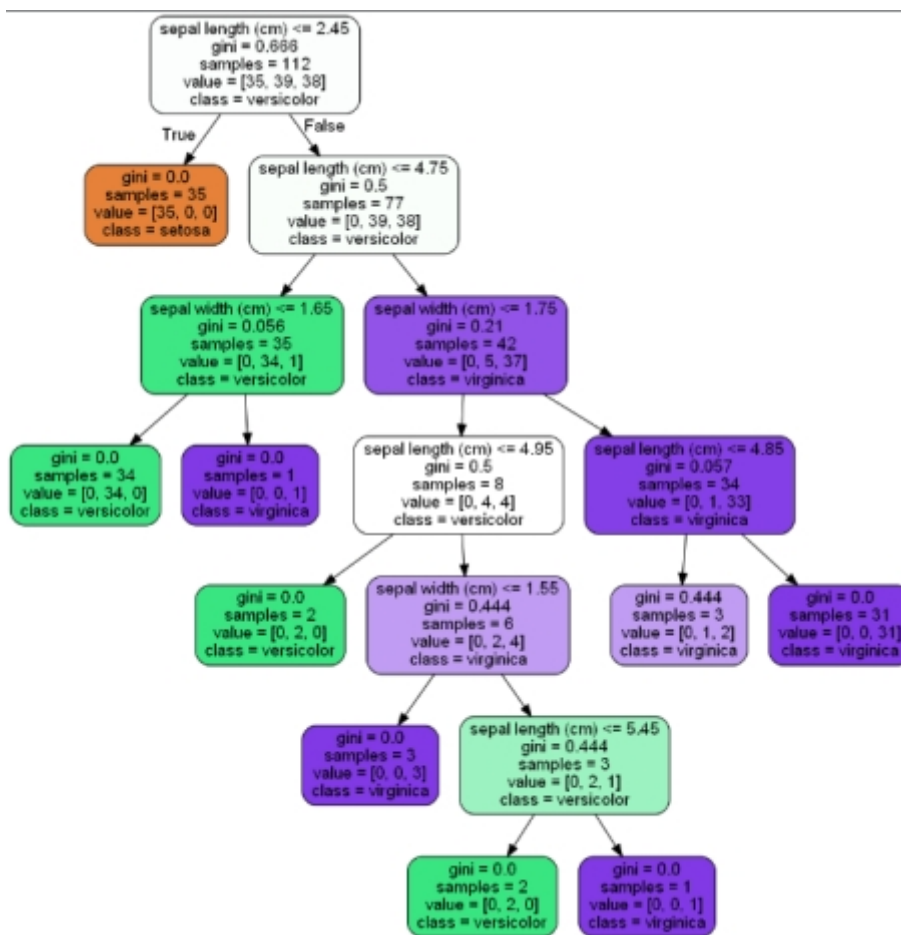
```
tree_clf = DecisionTreeClassifier(max_depth=8, criterion='gini')
tree_clf.fit(x_train, y_train)
```

graphviz强大而便捷的关系图/流程图绘制方法，很容易让我们联想到机器学习中的Decision Tree的展示方式。幸运的是，scikit-learn提供了生成.dot文件的接口

```
export_graphviz(  
    tree_clf,  
    out_file="./iris_tree.dot",  
    feature_names=iris.feature_names[:2],  
    class_names=iris.target_names,  
    rounded=True,  
    filled=True  
)
```

进入windows命令行界面，cd 切换到tree.dot所在的路径，执行

```
C:\Program Files (x86)\Graphviz2.38\bin>. \dot -Tpng D:\PythonProject\AI_Study\decision_tree\iris_tree.dot -o D:\PythonProject\AI_Study\decision_tree\iris_tree.png
```



对测试集进行评估

```
y_test_hat = tree_clf.predict(x_test)  
print("acc score:", accuracy_score(y_test, y_test_hat))
```

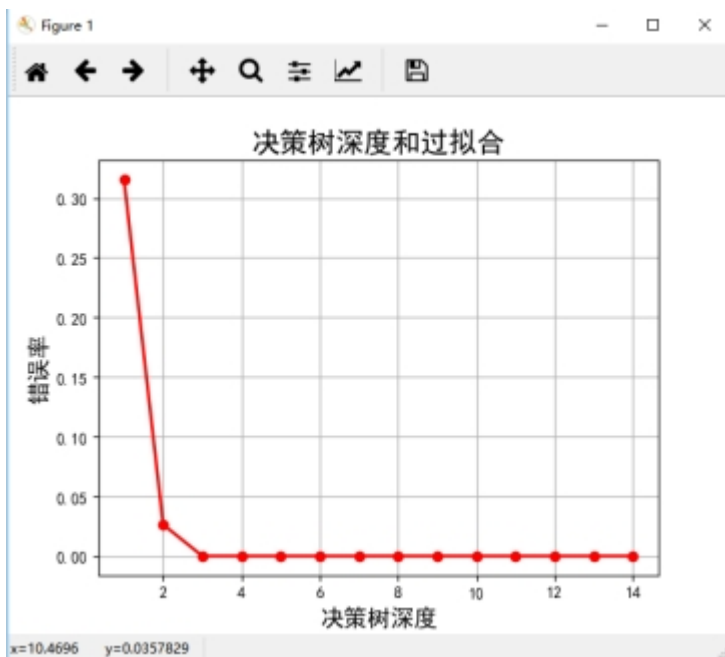
对新测试样本进行预测

```
print(tree_clf.predict_proba([[5, 1.5]]))  
print(tree_clf.predict([[5, 1.5]]))
```

```
[[0. 0. 1.]]  
[2]
```

绘制不同超参数对应决策树模型的图形

```
depth = np.arange(1, 15)  
err_list = []  
for d in depth:  
    print(d)  
    clf = DecisionTreeClassifier(criterion='gini', max_depth=d)  
    clf.fit(x_train, y_train)  
    y_test_hat = clf.predict(x_test)  
    result = (y_test_hat == y_test)  
    if d == 1:  
        print(result)  
    err = 1 - np.mean(result)  
    print(100 * err)  
    err_list.append(err)  
    print(d, ' 错误率: %.2f%%' % (100 * err))  
  
mpl.rcParams['font.sans-serif'] = ['SimHei']  
plt.figure(facecolor='w')  
plt.plot(depth, err_list, 'ro-', lw=2)  
plt.xlabel('决策树深度', fontsize=15)  
plt.ylabel('错误率', fontsize=15)  
plt.title('决策树深度和过拟合', fontsize=18)  
plt.grid(True)  
plt.show()
```



副产品

打印特征重要性评分

```
print(tree_clf.feature_importances_)
```

```
[0.91612234 0.08387766]
```

回归树

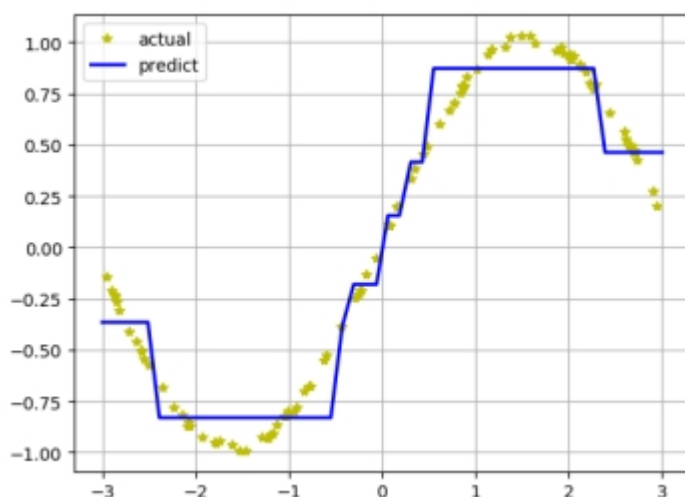
训练单颗回归树

```
import numpy as np
from sklearn.tree import DecisionTreeRegressor
import matplotlib.pyplot as plt

N = 100
x = np.random.rand(N) * 6 - 3
y = np.sin(x) + np.random.rand(N) * 0.05
x = x.reshape(-1, 1)

dt_reg = DecisionTreeRegressor(criterion='mse', max_depth=3)
dt_reg.fit(x, y)
x_test = np.linspace(-3, 3, 50).reshape(-1, 1)
y_hat = dt_reg.predict(x_test)

plt.plot(x, y, "y*", label="actual")
plt.plot(x_test, y_hat, "b-", linewidth=2, label="predict")
plt.legend(loc="upper left")
plt.grid()
plt.show()
# plt.savefig("./temp_decision_tree_regressor")
```



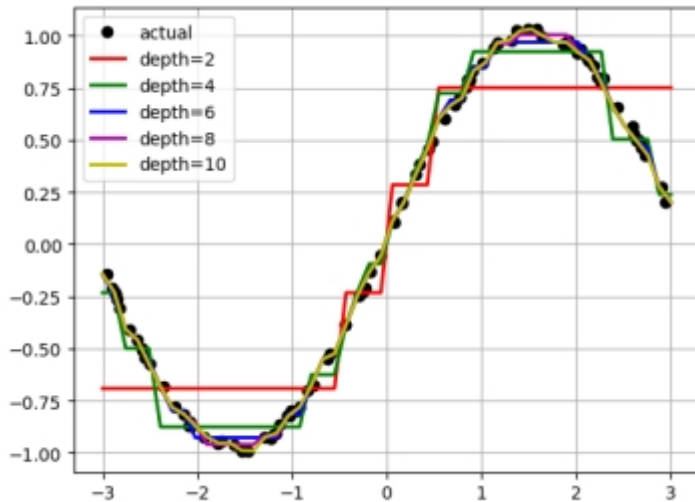
调整超参数对比

```
# 比较不同深度的决策树
depth = [2, 4, 6, 8, 10]
color = 'rgbmy'
dt_reg = DecisionTreeRegressor()
plt.plot(x, y, "ko", label="actual")
```

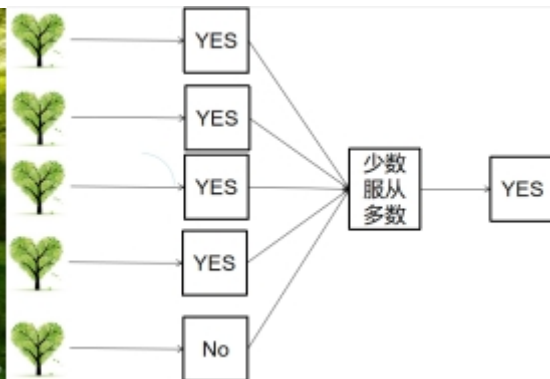
```

x_test = np.linspace(-3, 3, 50).reshape(-1, 1)
for d, c in zip(depth, color):
    dt_reg.set_params(max_depth=d)
    dt_reg.fit(x, y)
    y_hat = dt_reg.predict(x_test)
    plt.plot(x_test, y_hat, '-', color=c, linewidth=2, label="depth=%d" % d)
plt.legend(loc="upper left")
plt.grid(b=True)
plt.show()
# plt.savefig("./temp_compare_decision_tree_depth")

```



章节5：随机森林算法与集成学习



集成学习

假设你有15个朋友给你出主意投资股票，你怎么做最终的决定？

1. 选择最牛x的一个朋友的意见当作自己的意见（找到最好的单颗决策树）
2. 所有朋友的意见投票，少数服从多数（随机森林）
3. 还是投票牛一点的朋友多给几票，弱鸡一点的少给几票（Adaboost）
4. 买啥跌啥的朋友意见该怎么办？（使用技巧）

聚合模型 Aggregation Model

所有朋友的意见投票, 少数服从多数

$$G(\mathbf{x}) = \text{sign}\left(\sum_{t=1}^T \mathbf{1} \cdot g_t(\mathbf{x})\right) \quad G(\mathbf{x}) = \frac{1}{T} \sum_{t=1}^T g_t(\mathbf{x})$$

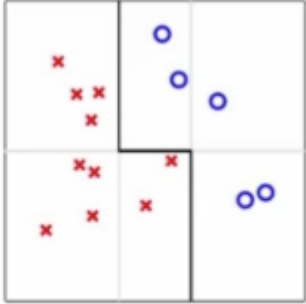
随机森林

Uniform Blending (Voting) for Classification

uniform blending: known g_t , each with 1 ballot

$$G(\mathbf{x}) = \text{sign}\left(\sum_{t=1}^T \mathbf{1} \cdot g_t(\mathbf{x})\right)$$

- same g_t (autocracy):
as good as one single g_t
- very different g_t (diversity + democracy):
majority can correct minority



还是投票牛x一点的朋友多给几票, 弱鸡一点的少给几票

$$G(\mathbf{x}) = \text{sign}\left(\sum_{t=1}^T \alpha_t \cdot g_t(\mathbf{x})\right) \text{ with } \alpha_t \geq 0$$

可是, 我没有朋友

制造一些朋友... $g(x)$, 每一个朋友好比是一个base model。

如果每个人的看法都一模一样 那没有投票的意义

制造出看法迥异 但是多少都有些道理的朋友

如何生成不一样的 $g(x)$

同样的数据, 行列都相同, 不同得超参数, 可以得到不同的模型。

同样的超参数, 行相同, 列不同, 可以得到不同的模型。

同样的超参数, 列相同, 行不同, 可以得到不同的模型。

同样的超参数, 同样的数据, 但是数据权重不同, 可以得到不同的模型。

Bagging

对训练集进行抽样, 将抽样的结果用于训练 $g(x)$ 。并行, 独立训练。random forest

Boosting

利用训练集训练出模型 根据本次模型的预测结果, 调整训练集, 然后利用

调整后的训练集训练下一个模型。串行, 需要第一个模型。adaboost, GBDT, Xgboost

Bagging

Bootstrap : 有放回地对原始数据集进行均匀抽样

利用每次抽样生成的数据集训练模型

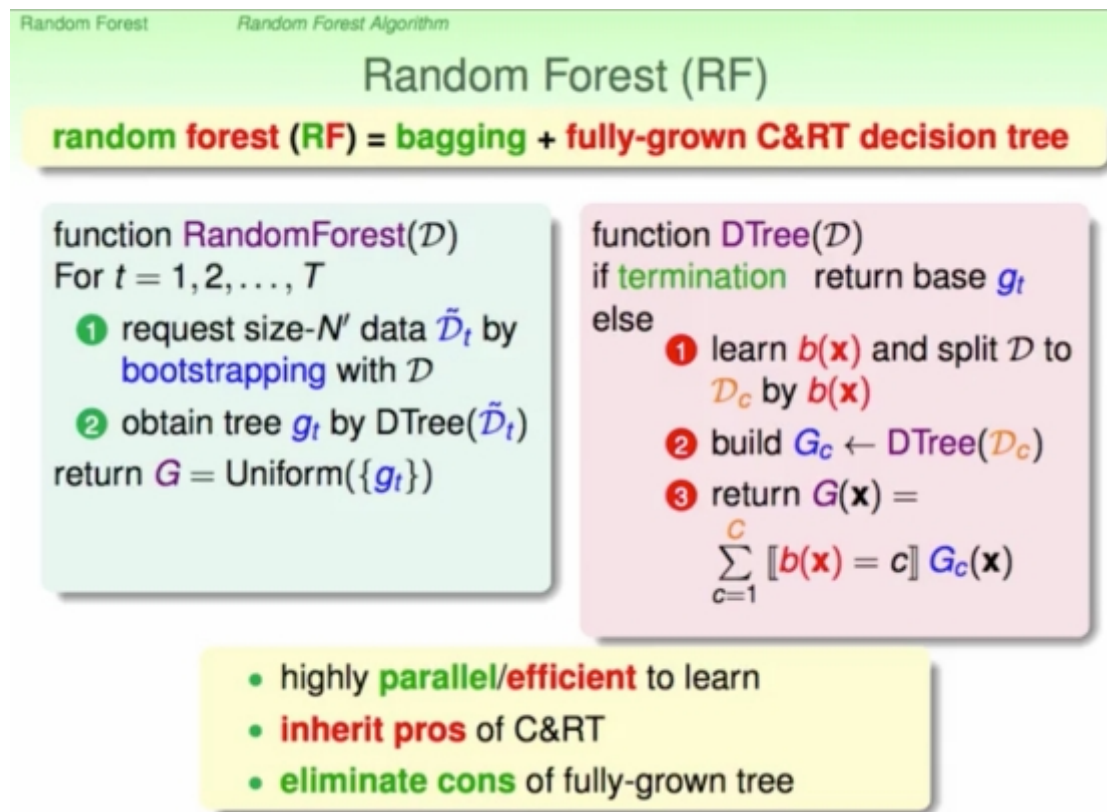
最终的模型为每次生成的模型进行投票

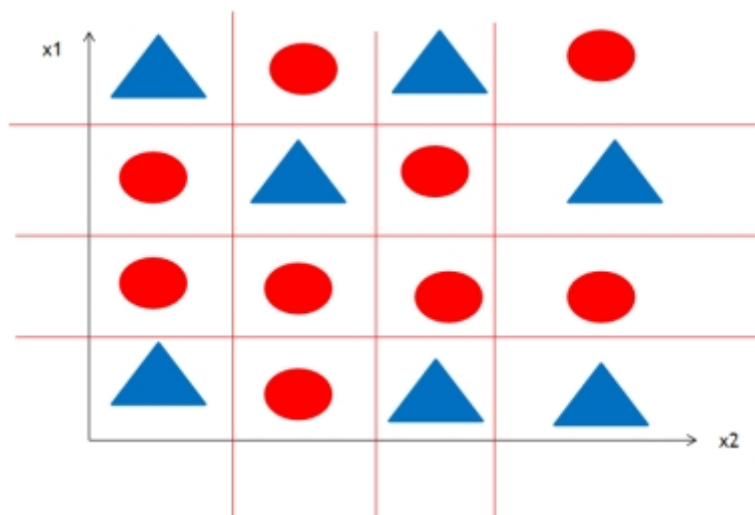
其实boosting和bagging都不仅局限于对决策树这种基模型适应

如果不是同一种base model, stacking

随机森林

Bagging思想+决策树作为base model+uniform blending

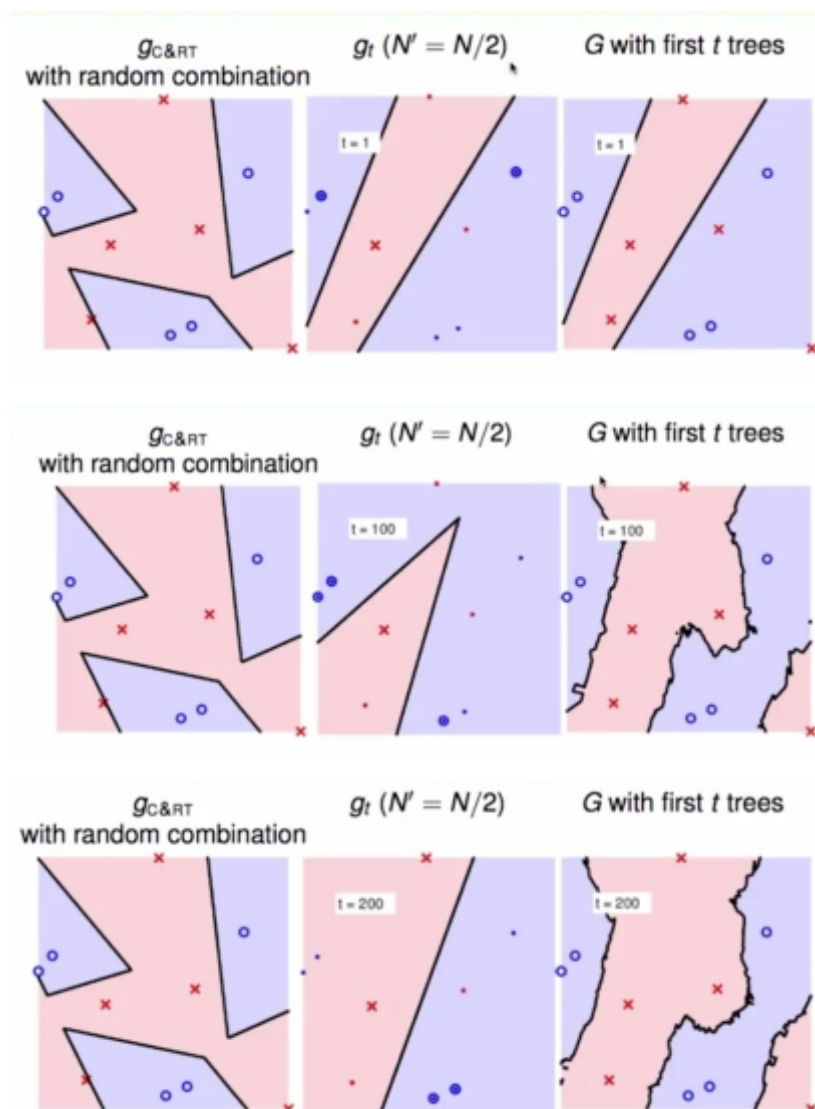


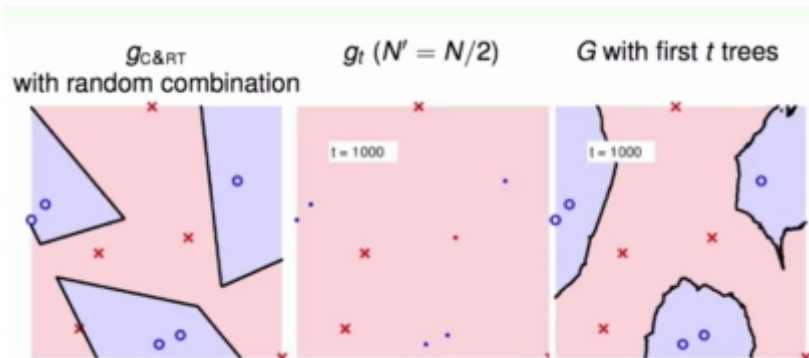
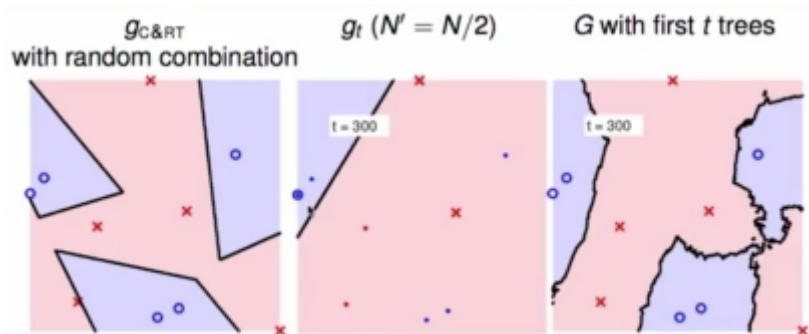


都有哪些随机?

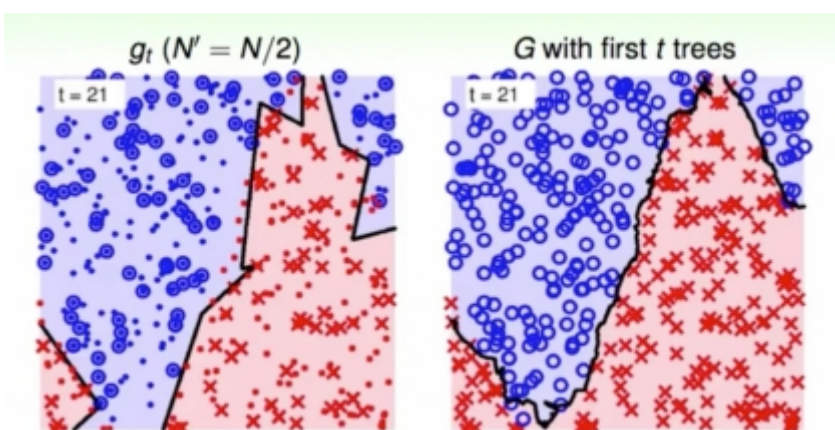
1. bagging生成小树时随机抽样数据
2. 抽样数据后 对feature也进行抽样

随机森林表现

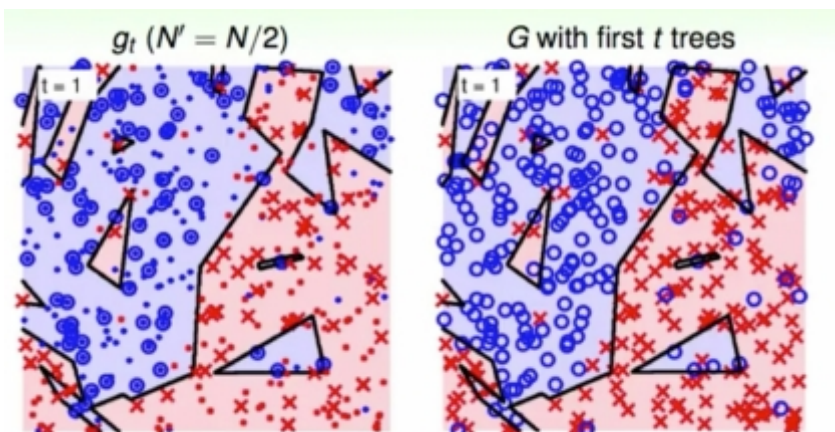


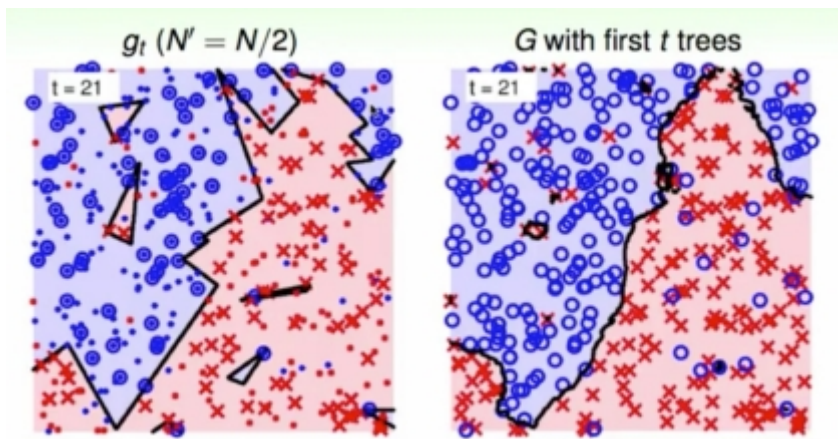


复杂数据表现



有噪声数据表现





总结

随机森林

- 1、随机选择样本（放回抽样）；
- 2、随机选择特征；
- 3、构建决策树；
- 4、随机森林投票（平均）

优点:

1. 表现良好
2. 可以处理高维度数据(维度随机选择)
3. 辅助进行特征选择
4. 得益于bagging 可以进行并行训练

缺点:

对于噪声过大的数据容易过拟合

代码实战

训练模型

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_iris

iris = load_iris()
X = iris.data[:, :2] # 花萼长度和宽度
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33,
                                                    random_state=42)

rnd_clf = RandomForestClassifier(n_estimators=5, max_leaf_nodes=16, n_jobs=1,
                                oob_score=True)
rnd_clf.fit(X_train, y_train)

y_pred_rf = rnd_clf.predict(X_test)
```

```
print(accuracy_score(y_test, y_pred_rf))

# Feature Importance
iris = load_iris()
rnd_clf = RandomForestClassifier(n_estimators=500, n_jobs=-1)
rnd_clf.fit(iris["data"], iris['target'])
for name, score in zip(iris['feature_names'], rnd_clf.feature_importances_):
    print(name, score)
```

Out of bag data

在进行抽样的时候, 有很多条数据并没有被选到

数量:

1条数据N轮没有备选到的概率:

$$\left(1 - \frac{1}{N}\right)^N$$

当N足够大的时候:

$$\left(1 - \frac{1}{N}\right)^N = \frac{1}{\left(\frac{N}{N-1}\right)^N} = \frac{1}{\left(1 + \frac{1}{N-1}\right)^N} \approx \frac{1}{e}$$

	g_1	g_2	g_3	...	g_T
$(\mathbf{x}_1, y_1)$	$\tilde{\mathcal{D}}_1$	★	$\tilde{\mathcal{D}}_3$		$\tilde{\mathcal{D}}_T$
$(\mathbf{x}_2, y_2)$	★	★	$\tilde{\mathcal{D}}_3$		$\tilde{\mathcal{D}}_T$
$(\mathbf{x}_3, y_3)$	★	$\tilde{\mathcal{D}}_2$	★		$\tilde{\mathcal{D}}_T$
...					
$(\mathbf{x}_N, y_N)$	$\tilde{\mathcal{D}}_1$	$\tilde{\mathcal{D}}_2$	★		★

可不可以使用OOB数据来当作验证集呢?

使用OOB验证g? 不太需要

使用OOB验证G? How?

OOB versus Validation

OOB

	g_1	g_2	g_3	...	g_T
$(\mathbf{x}_1, y_1)$	$\tilde{D}_1$	*	$\tilde{D}_3$		$\tilde{D}_T$
$(\mathbf{x}_2, y_2)$	*	*	$\tilde{D}_3$		$\tilde{D}_T$
$(\mathbf{x}_3, y_3)$	*	$\tilde{D}_2$	*		$\tilde{D}_T$
...					
$(\mathbf{x}_N, y_N)$	$\tilde{D}_1$	*	*		*

Validation

g_1^-	g_2^-	...	g_M^-
$\mathcal{D}_{\text{train}}$	$\mathcal{D}_{\text{train}}$		$\mathcal{D}_{\text{train}}$
$\mathcal{D}_{\text{val}}$	$\mathcal{D}_{\text{val}}$		$\mathcal{D}_{\text{val}}$
$\mathcal{D}_{\text{val}}$	$\mathcal{D}_{\text{val}}$		$\mathcal{D}_{\text{val}}$
$\mathcal{D}_{\text{train}}$	$\mathcal{D}_{\text{train}}$		$\mathcal{D}_{\text{train}}$

- * like $\mathcal{D}_{\text{val}}$: 'enough' random examples unused during training
- use * to validate g_t ? easy, but **rarely needed**
- use * to validate G ? $E_{\text{oob}}(G) = \frac{1}{N} \sum_{n=1}^N \text{err}(y_n, G_n^-(\mathbf{x}_n))$,
with G_n^- contains only trees that $\mathbf{x}_n$ is OOB of,
such as $G_N^-(\mathbf{x}) = \text{average}(g_2, g_3, g_T)$

E_{oob} : self-validation of bagging/RF

```
print(rnd_clf.oob_score_)
```

章节6: Adaboost算法

Blending

Blending known diverse hypotheses uniformly, linearly, non-linearly

v uniformly blending -- Bagging, weight is 1

$$G(x) = \text{sign}(\sum g(x))$$

v linearly blending -- Boosting, weight is unequal

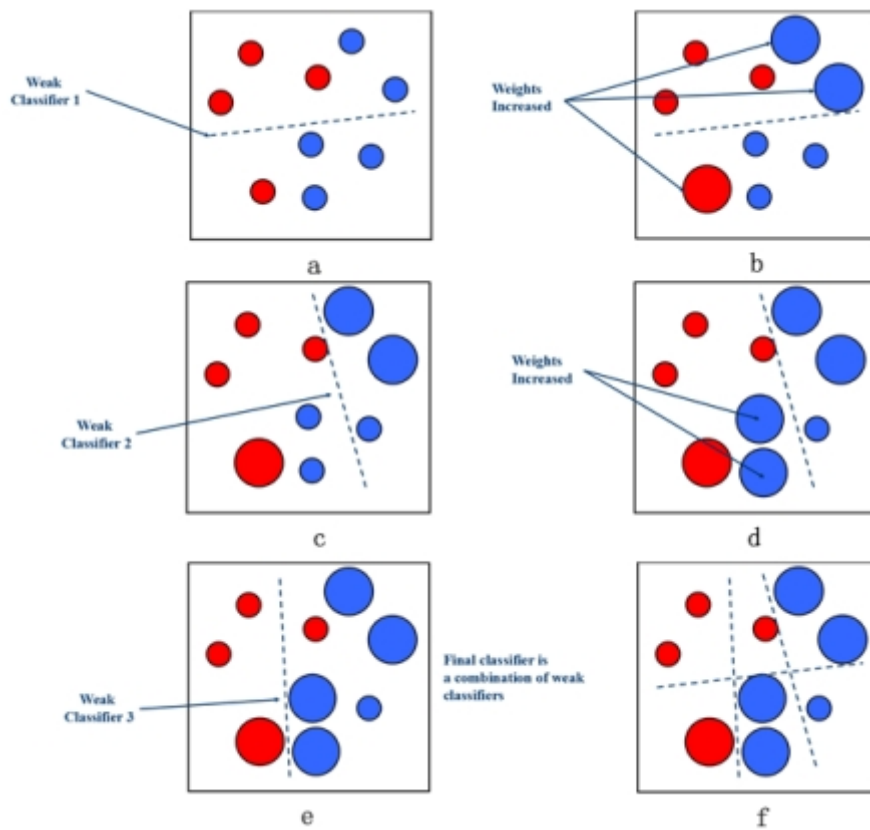
$$G(x) = \text{sign}(\sum \alpha \cdot g(x))$$

v non-linearly blending -- Boosting

$$G(x) = \text{sign}(\sum h(g(x)))$$

Boosting

- ☐ 集成学习的一种思路
- ☐ 多轮迭代 (全部数据) 每一轮产生一个弱分类器
- ☐ 将弱分类器预测结果相加, 得到强分类器的最终结果
- ☐ Adaboost、GBDT、Xgboost
- ☒ RF、Adaboost、GBDT和XGBoost都属于集成学习 (Ensemble Learning), 集成学习的目的是通过结合多个基学习器的预测结果来改善单个学习器的泛化能力和鲁棒性



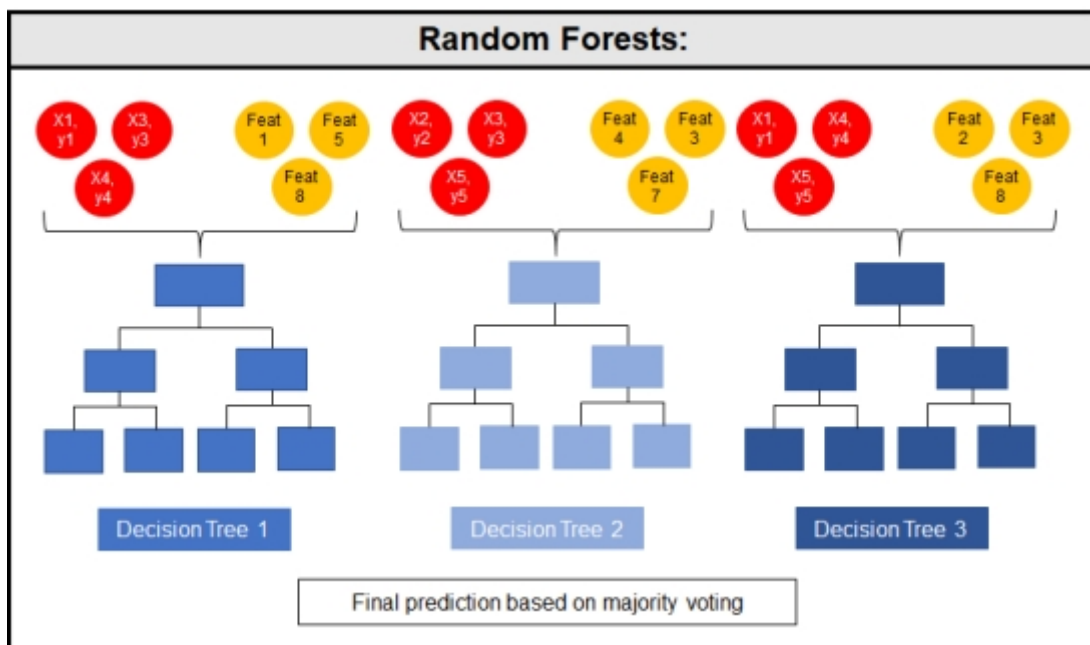
什么是Adaboost

- 对样本赋予权重，采用迭代的方式来构造每一步的弱分类器
- 最终通过弱分类器的linear blending 得到强分类器
 - $G(x) = \alpha_1 * g_1(x) + \alpha_2 * g_2(x) + \alpha_3 * g_3(x) + \dots + \alpha_n * g_n(x)$
 - $= \alpha \cdot g(x)$ (向量形式)
- 待解决问题:
 - 如何迭代生成 若干互不相同的 $g(x)$
 - 如何获得 n 个 α

如何获得不同的 $g(x)$

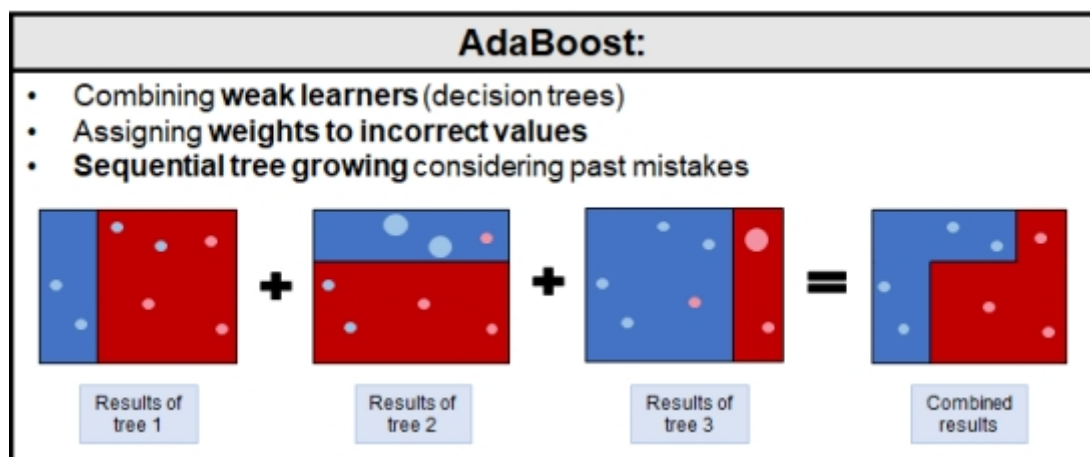
随机森林

- ✓ bootstrap re-sample
- ✓ $g(x)$ 什么特点?
- ✓ 如何利用 $g(x)$ 生成 $G(x)$



AdaBoost

- 通过赋予样本不同的权重获得不同的 $g(x)$
- 基分类器 $g(x)$ 大多使用decision-stump决策桩
- 我们通过修改数据的权重，使得本次训练的弱分类器在上次弱分类器做的不好的地方进行训练



什么是数据的权重weight

bagging中权重是什么意思？

$$\mathcal{D} = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), (\mathbf{x}_3, y_3), (\mathbf{x}_4, y_4)\}$$

bootstrap $\Rightarrow$

$$\tilde{\mathcal{D}}_t = \{(\mathbf{x}_1, y_1), (\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), (\mathbf{x}_4, y_4)\}$$

weighted E_{in} on $\mathcal{D}$

$$E_{in}^u(h) = \frac{1}{4} \sum_{n=1}^4 u_n^{(t)} \cdot \mathbb{I}[y_n \neq h(\mathbf{x}_n)]$$

$(\mathbf{x}_1, y_1), u_1 = 2$
 $(\mathbf{x}_2, y_2), u_2 = 1$
 $(\mathbf{x}_3, y_3), u_3 = 0$
 $(\mathbf{x}_4, y_4), u_4 = 1$

E_{in} on $\tilde{\mathcal{D}}_t$

$$E_{in}^{0/1}(h) = \frac{1}{4} \sum_{(\mathbf{x}, y) \in \tilde{\mathcal{D}}_t} \mathbb{I}[y \neq h(\mathbf{x})]$$

$(\mathbf{x}_1, y_1), (\mathbf{x}_1, y_1)$
 $(\mathbf{x}_2, y_2)$

 $(\mathbf{x}_4, y_4)$

权重与函数的关系

对于同一个算法

- 训练集不同，生成的模型一定不同
- 如果训练集相同，我们调整训练集中数据的权重，生成的模型也一定不同

对于同一个模型

- 输入的数据权重不同，模型预测的正确率也一定不同
- 可以通过调整输入数据的权重，让本来还不错的分类器的正确率达到1/2

Adaboost中的数据权重 u_n

定义

数据预测的错误率 = (错误数据的个数) / (全部数据的个数)

数据预测的权重错误率 = (预测错误数据的权重和) / (全部数据的权重和)

举例

有一个标签集实际是{+1, +1, +1, +1, -1}

- 我们训练出一个 $g_1(x)$ 的分类结果{+1, +1, +1, +1, +1}
- 当所有数据的权重都为1/5时，错误率是多少？权重错误率是多少？
- 当-1的权重是1/2，其余+1是1/8的时候，权重错误率是多少？

小结

- 在训练集的每个数据背后都标注一个权重，初始权重为1/N，此时数据错误率等于权重错误率
- 每一轮训练一个当前权重下权重错误率最低的 $g(x)$
- 迭代生成下一轮的权重
- 考虑一下 decision-stump，下一轮数据权重变了，训练出来的分类器内部会发生什么改变？

具体实现流程

既然通过权重不同来训练弱分类器模型 $g_t(x)$ ，那么我们每次找到 $g(x)$ 都应该使当前时刻权重正确率最大

目标

$$g_t \leftarrow \operatorname{argmin}_{h \in \mathcal{H}} \left(\sum_{n=1}^N u_n^{(t)} \mathbb{I}[y_n \neq h(\mathbf{x}_n)] \right)$$

$$g_{t+1} \leftarrow \operatorname{argmin}_{h \in \mathcal{H}} \left(\sum_{n=1}^N u_n^{(t+1)} \mathbb{I}[y_n \neq h(\mathbf{x}_n)] \right)$$

现在我们根据 U_n^t 得到了当前权重下表现最好的弱分类器 $g_t(x)$, 接下来想要得到下一轮的 U_n^{t+1} 算法才能继续工作下去

如何计算 U_n^{t+1} ?

我们想要通过 U_n^{t+1} 训练若分类器 $g_{t+1}(x)$, 想要通过 U_n^{t+1} 赋予 $g_{t+1}(x)$ 在之前 $g_t(x)$ 表现不好的地方得到补强

want: $\frac{\sum_{n=1}^N u_n^{(t+1)} \mathbb{I}[y_n \neq g_t(\mathbf{x}_n)]}{\sum_{n=1}^N u_n^{(t+1)}} = \frac{\blacksquare_{t+1}}{\blacksquare_{t+1} + \bullet_{t+1}} = \frac{1}{2}$, where

$$\blacksquare_{t+1} = \sum_{n=1}^N u_n^{(t+1)} \mathbb{I}[y_n \neq g_t(\mathbf{x}_n)], \bullet_{t+1} = \sum_{n=1}^N u_n^{(t+1)} \mathbb{I}[y_n = g_t(\mathbf{x}_n)]$$

- need: $\underbrace{(\text{total } u_n^{(t+1)} \text{ of incorrect})}_{\blacksquare_{t+1}} = \underbrace{(\text{total } u_n^{(t+1)} \text{ of correct})}_{\bullet_{t+1}}$
- one possibility by **re-scaling (multiplying) weights**, if

(total $u_n^{(t)}$ of incorrect) = 1126 ;	(total $u_n^{(t)}$ of correct) = 6211 ;
(weighted incorrect rate) = $\frac{1126}{7337}$	(weighted correct rate) = $\frac{6211}{7337}$
incorrect: $u_n^{(t+1)} \leftarrow u_n^{(t)} \cdot 6211$	correct: $u_n^{(t+1)} \leftarrow u_n^{(t)} \cdot 1126$

'optimal' re-weighting under weighted incorrect rate ϵ_t :
multiply incorrect $\propto (1 - \epsilon_t)$; multiply correct $\propto \epsilon_t$

迭代每一轮物理权重 $u_n^{(t+1)}$ 的方式

define scaling factor $\diamond_t = \sqrt{\frac{1-\epsilon_t}{\epsilon_t}}$

$$\begin{aligned} \text{incorrect} &\leftarrow \text{incorrect} \cdot \diamond_t \\ \text{correct} &\leftarrow \text{correct} / \diamond_t \end{aligned}$$

- equivalent to optimal re-weighting
- $\diamond_t \geq 1$ iff $\epsilon_t \leq \frac{1}{2}$
—physical meaning: **scale up incorrect**; **scale down correct**

设A 为正确的总权重和 B为错误的总权重和:

$$\text{有 } A\epsilon = B(1-\epsilon)$$

$$\text{总权重和 } C = A/(1-\epsilon) = B/\epsilon$$

$$A\epsilon A/(1-\epsilon) = B(1-\epsilon)B/\epsilon$$

$$A\epsilon/(1-\epsilon) = B*B*(1-\epsilon)/\epsilon$$

$$A*\sqrt{\epsilon/(1-\epsilon)} = B*\sqrt{(1-\epsilon)/\epsilon}$$

需要弱一点的树

砍树+抽样

Weak Decision Tree Algorithm

AdaBoost: **votes** $\alpha_t = \ln \diamond_t = \ln \sqrt{\frac{1-\epsilon_t}{\epsilon_t}}$ with **weighted error rate** ϵ_t

if **fully grown** tree trained on **all** $\mathbf{x}_n$
 $\Rightarrow E_{in}(g_t) = 0$ if **all** $\mathbf{x}_n$ different
 $\Rightarrow E_{in}^u(g_t) = 0$
 $\Rightarrow \epsilon_t = 0$
 $\Rightarrow \alpha_t = \infty$ (**autocracy!!**)

need: **pruned** tree trained on **some** $\mathbf{x}_n$ to be **weak**

- **pruned**: usual pruning, or just **limiting tree height**
- **some**: **sampling** $\propto u^{(t)}$

否则独裁

AdaBoost-DTree: often via AdaBoost +
sampling $\propto u^{(t)}$ + **pruned** DTree($\tilde{D}$)

合并生成G(x)

线性聚合

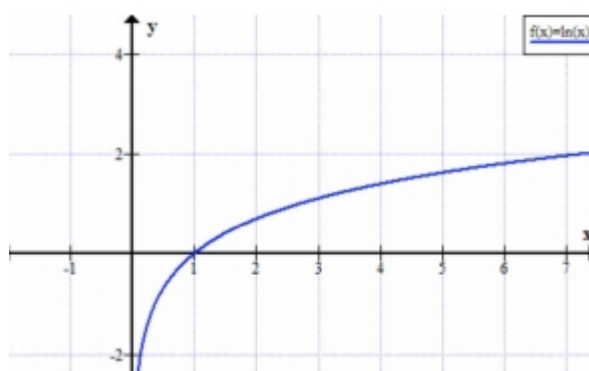
$$G(\mathbf{x}) = \text{sign} \left(\sum_{t=1}^T \alpha_t g_t(\mathbf{x}) \right)$$

计算 α 的方法

对于优秀的g(x)应该给与的权重大

$$\diamond_t = \sqrt{\frac{1-\epsilon_t}{\epsilon_t}}$$

找到与 Δt 正相关且 Δt 为1时候值为0的 α



$$\alpha = \frac{1}{2} \ln\left(\frac{1-\varepsilon}{\varepsilon}\right)$$

$$\alpha = \ln(\Delta t)$$

Adaboost 全局过程

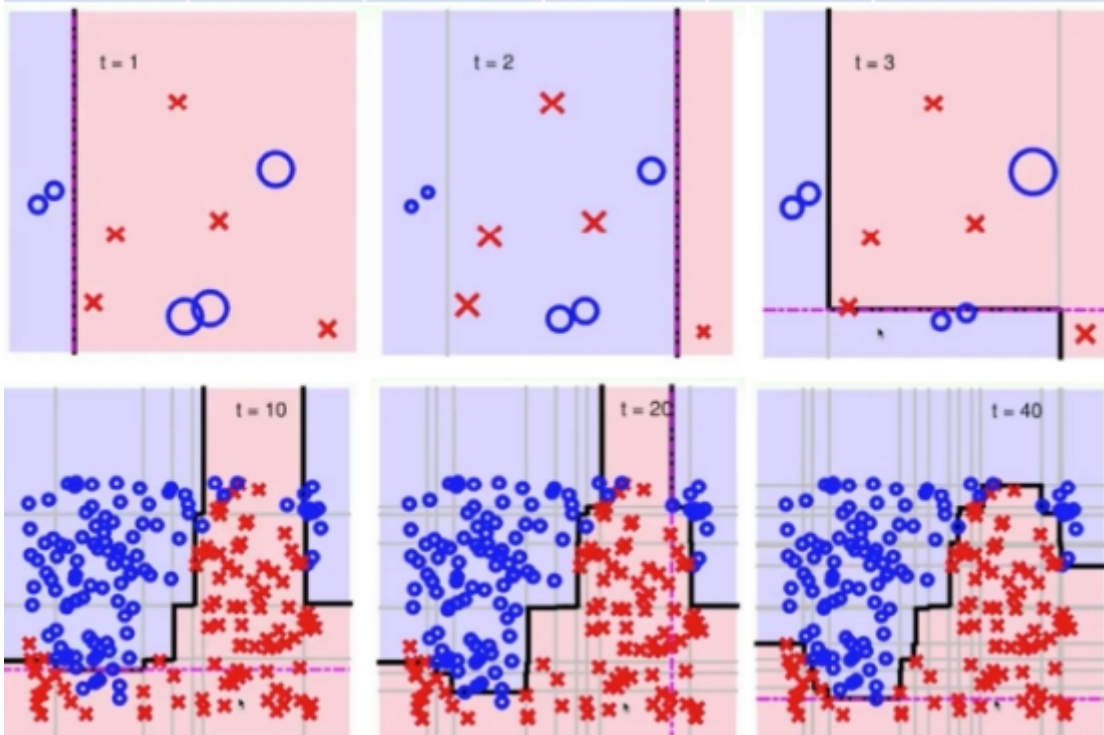
1. 初始化权重 u_1

2. 开始迭代:

- 2.1 根据 u_t 训练模型 g_t
- 2.2 根据 g_t 模型在 u_t 数据上的结果 计算 中间结果 $\diamond_t$
- 2.3 根据 $\diamond_t$ 计算下一轮数据权重 u_{t+1}
- 2.4 根据 $\diamond_t$ 计算本轮弱分类器权重 α_t

3. 迭代完成后 得到一组 $g(x)$ 返回: $G(\mathbf{x}) = \text{sign}\left(\sum_{t=1}^T \alpha_t g_t(\mathbf{x})\right)$

	样本1	样本2	样本3	样本4	
u_1	1	1	1	1	
$[[y_n \neq g_t(x_n)]]$	0	0	1	0	$\varepsilon_1 = 0.25$ $\diamond_t = \sqrt{\frac{1-\varepsilon_t}{\varepsilon_t}} = \sqrt{3}$ $\alpha_1 = \ln\sqrt{3}$
u_2	$\frac{1}{\sqrt{3}}$	$\frac{1}{\sqrt{3}}$	$\sqrt{3}$	$\frac{1}{\sqrt{3}}$	$\varepsilon'_1 = 0.5$



Adaboost 总结

- 把弱 聚合 强
- 每次生成新的权重时放大上一次错的 缩小上一次对的
- 把得到的 $g(x)$ 线性组合得到最后的 $G(x)$

章节7：GBDT梯度提升决策树

回顾有监督机器学习

假定有N个训练样本： $\{(X(1), y(1)), (X(2), y(2)), \dots, (X(n), y(n))\}$

找到一个函数 $F(x)$ 对应一种映射使得其损失函数最小

$$F^* = \underset{F(X)}{\operatorname{argmin}} L(y, F(X))$$

如何最小？

回顾梯度下降

$$F^* = \underset{F(X)}{\operatorname{argmin}} L(y, F(X))$$

$$F(X; W) = WX = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$$

找到一组W 使得 L 最小 进而求得F*

找到下降方向 $d_i = -\frac{\partial}{\partial w} f(w)|_{w_i}$ 不断更新 $w_{i+1} = w_i + \rho \cdot d_i$

当最终结果保持不变时 认为我们找到了使 $f(w)$ 也就是 $L(y, F(x))$ 最小的一组 w

最终求得的 最终求得的 w 可以表示为连加的结构

$$w^* = \sum_{i=0}^M \rho_i \cdot d_i$$

函数空间的梯度下降

类似于向量空间的梯度下降, 既然损失函数可表示为: $L(y, F(X))$

直接将 $F(x)$ 看做梯度下降中的 w, 每次迭代过程中让 $L(y, F(X))$ 对 $F(x)$ 求偏导得到:

$$g_m(X) = \left[\frac{\partial L(y, F(X))}{\partial F(X)} \right]_{F(X)=F_{m-1}(X)}$$

每次得到的函数的偏移量: $f_i(x) = -\rho_i g_m(x)$

从初始的 $F_0(x)$ 每一步迭代加上上面的函数偏移量 最终得到的函数也应该为能够使得误差函数 $L(y, F(X))$ 最小的函数

加法形式:

$$F_m(X) = \sum_{i=0}^m f_i(X)$$

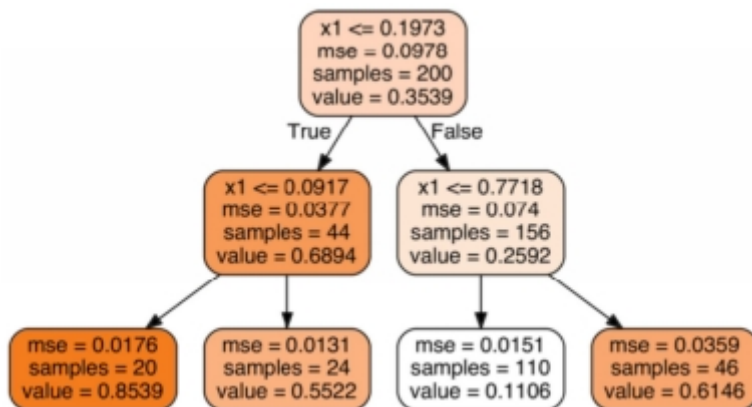
	向量空间的梯度下降 GradientDescent	函数空间的梯度下降 GradientBoosting
优化的内容	优化向量w	优化函数F(x)
为何优化内容不同	如LR和线性回归，是有w的，可以优化w，得到model	如DT，没有w，只能对整体F(x)求导，认为模型训练内部是黑盒子，只能改变y，输入给训练找到合适的model
公式	$d_i = -\frac{\partial}{\partial w} f(w) _{w_i}$	$g_m(X) = \left[\frac{\partial L(y, F(X))}{\partial F(X)} \right]_{F(X)=F_{m-1}(X)}$
迭代的公式：	$w_{i+1} = w_i + \rho \cdot d_i$	$f_i(x) = f_{i-1}(x) - \rho_i g_m(x)$

GBDT（梯度提升决策树）

- 利用多棵回归树处理回归问题的解决方案，通过调整损失函数也可以处理分类问题
- 通过全部样本迭代生成多棵回归树，对于回归问题的损失函数是什么？

回归树

- 每次分裂后计算分裂出的节点的平均值
- 将平均值带入MSE损失函数进行评估
- 分裂标准：1、分裂特征 2、分裂的值



每颗回归树拟合负梯度

GBDT 应用于回归问题

回归树 损失函数定义为 $\sum (y - f(x))^2$

负梯度:

Setting	Loss Function	$-\partial L(y_i, f(x_i)) / \partial f(x_i)$
Regression	$\frac{1}{2} [y_i - f(x_i)]^2$	$y_i - f(x_i)$

希望每次的gt(x)=残差 $y_i - f(x_i)$

使用残差作为标签训练一颗新的决策树

整体流程

(1) 初始化 $f_0(x) = 0$

(2) 对 $m = 1, 2, \dots, M$

(a) 计算残差

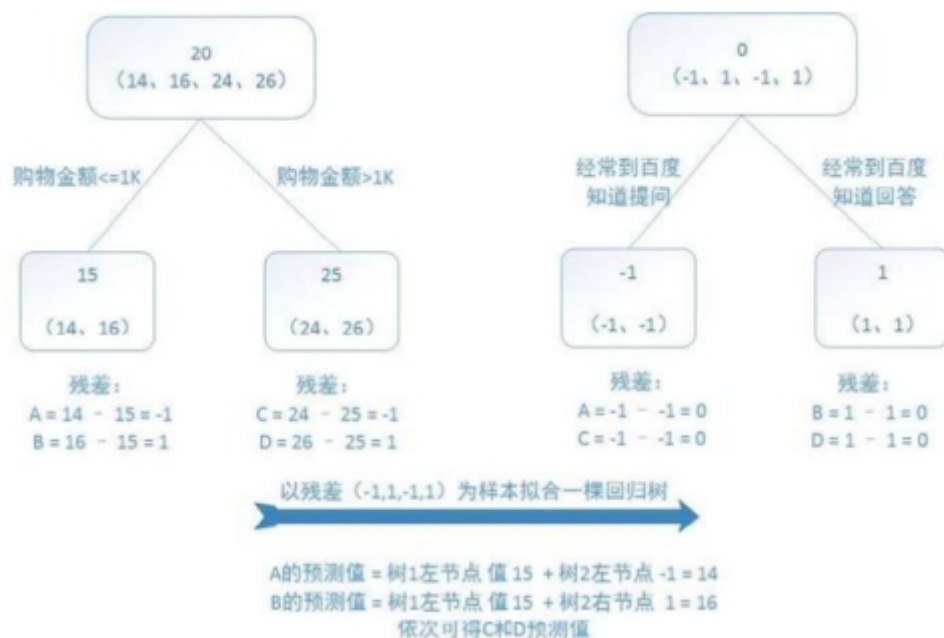
$$r_{mi} = y_i - f_{m-1}(x_i), i = 1, 2, \dots, N$$

(b) 拟合残差 r_{mi} 学习一个回归树，得到 $T(x; \phi_m)$

(c) 更新 $f_m(x) = f_{m-1}(x) + T(x; \phi_m)$

(3) 得到回归问题提升树
$$f_M(x) = \sum_{m=1}^M T(x; \phi_m)$$

举个例子



总结

GBDT与传统的Boosting区别较大，它的每一次计算都是为了减少上一次的残差，而为了消除残差，我们可以在残差减小的梯度方向上建立模型，所以说，在GradientBoost中，每个新的模型的建立是为了使得之前的模型的残差往梯度下降的方法，与传统的Boosting中关注正确错误的样本加权有着很大的区别。

在GradientBoosting算法中，关键就是利用损失函数的负梯度方向在当前模型的值作为残差的近似值，进而拟合一棵CART回归树。

GBDT的会累加所有树的结果，而这种累加是无法通过分类完成的，因此GBDT的树都是CART回归树，而不是分类树（尽管GBDT调整后也可以用于分类但不代表GBDT的树为分类树）。

章节8：应用GBDT做分类任务

GBDT应用于分类

GBDT算法概述

GBDT是boosting算法的一种，按照boosting的思想，在GBDT算法的每一步，用一棵决策树去拟合当前学习器的残差，获得一个新的弱学习器。将这每一步的决策树组合起来，就得到了一个强学习器。

具体来说，假设有训练样本 $\{x_i, y_i\}, i = 1 \dots n$ ，第m-1步获得的集成学习器为 $F_{m-1}(x)$ ，那么GBDT通过下面的递推式，获得一个新的弱学习器 $h(x)$ ：

$$F_m(x) = F_{m-1}(x) + \operatorname{argmin}_{h \in H} \sum_{i=1}^n \text{Loss}(y_i, F_{m-1}(x_i) + h(x_i))$$

其中 $h(x)$ 是在函数空间 H 上最小化损失函数，一般来说这是比较难以做到的。但是，如果我们只考虑精确地拟合训练数据的话，可以将损失函数 $\sum_{i=1}^n \text{Loss}(y_i, F_{m-1}(x_i) + h(x_i))$ 看作向量 $(F(x_1), F(x_2), \dots, F(x_n))$ 上的函数。这样在第m-1轮迭代之后，向量位于 $(F_{m-1}(x_1), F_{m-1}(x_2), \dots, F_{m-1}(x_n))$ ，如果我们想进一步减小损失函数，则根据梯度下降法，向量移动的方向应为损失函数的负梯度方向，即：

$$\mathbf{v} = - \left(\frac{\partial \text{loss}(y_1, F_{m-1}(x_1))}{\partial F_{m-1}(x_1)}, \frac{\partial \text{loss}(y_2, F_{m-1}(x_2))}{\partial F_{m-1}(x_2)}, \dots, \frac{\partial \text{loss}(y_n, F_{m-1}(x_n))}{\partial F_{m-1}(x_n)} \right)$$

这样如果使用训练集： $\left\{ x_i, -\frac{\partial \text{loss}(y_i, F_{m-1}(x_i))}{\partial F_{m-1}(x_i)} \right\}_{i=1}^n$ 去训练一棵树的话，就相当于朝着损失函数减小的方向又走了一步（当然在实际应用中需要shrinkage，也就是考虑学习率）。由此可见，GBDT在本质上还是梯度下降法，每一步通过学习一棵拟合负梯度（也就是所谓“残差”）的树，来使损失函数逐渐减小。

二分类问题

将GBDT应用于回归问题，相对来说比较容易理解。因为回归问题的损失函数一般为平方差损失函数，这时的残差，恰好等于预测值与实际值之间的差值。每次拿一棵决策树去拟合这个差值，使得残差越来越小，这个过程还是比较intuitive的。而将GBDT用于分类问题，则显得不那么显而易见。下面我们就通过一个简单的二分类问题，去看看GBDT究竟是如何学习到一棵树的。

类似于逻辑回归、FM模型用于分类问题，其实是在用一个线性模型或者包含交叉项的非线性模型，去拟合所谓的对数几率 $\ln \frac{p}{1-p}$ 。而GBDT也是一样，只是用一系列的梯度提升树去拟合这个对数几率，实际上最终得到的是一系列CART回归树。其分类模型可以表达为：

$$P(y = 1|x) = \frac{1}{1 + e^{-\sum_{m=0}^M h_m(x)}}$$

其中 $h_m(x)$ 就是学习到的决策树。

清楚了这一点之后，我们便可以参考逻辑回归，单样本 (x_i, y_i) 的损失函数可以表达为交叉熵：

$$\text{loss}(x_i, y_i) = -y_i \log \hat{y}_i - (1 - y_i) \log(1 - \hat{y}_i)$$

假设第k步迭代之后当前学习器为 $F(x) = \sum_{m=0}^k h_m(x)$ ，将 $\hat{y}_i$ 的表达式带入之后，可将损失函数写为：

$$\text{loss}(x_i, y_i | F(x)) = y_i \log(1 + e^{-F(x_i)}) + (1 - y_i) [F(x_i) + \log(1 + e^{-F(x_i)})]$$

可以求得损失函数相对于当前学习器的负梯度为：

$$-\frac{\partial loss}{\partial F(x)}|_{x_i, y_i} = y_i - \frac{1}{1 + e^{-F(x_i)}} = y_i - \hat{y}_i$$

可以看到，同回归问题很类似，下一棵决策树的训练样本为： $\{x_i, y_i - \hat{y}_i\}_{i=1}^n$ ，其所需要拟合的残差为真实标签与预测概率之差。于是便有下面GBDT应用于二分类的算法：

1. $F_0(x) = h_0(x) = \log \frac{p_1}{1 - p_1}$ ，其中 p_1 是训练样本中 $y=1$ 的比例，利用先验信息来初始化学习器
2. For $m = 1, 2, \dots, M$:
3. 计算 $g_i = \hat{y}_i - y_i$ ，并使用训练集 $\{(x_i, -g_i)\}_{i=1}^n$ 训练一棵回归树 $t_m(x)$ ，其中 $\hat{y}_i = \frac{1}{1 + e^{-F_{m-1}(x)}}$
4. 通过一维最小化损失函数找到树的最优权重： $\rho_m = \operatorname{argmin}_{\rho} \sum_i loss(x_i, y_i | F_{m-1}(x) + \rho t_m(x))$
5. 考虑shrinkage，可得这一轮迭代之后的学习器 $F_m(x) = F_{m-1}(x) + \alpha \rho_m t_m(x)$ ， α 为学习率
6. 得到最终学习器为： $F_M(x)$

多分类问题

类似地，对于多分类问题，则需要考虑以下softmax模型：

$$P(y = 1|x) = \frac{e^{F_1(x)}}{\sum_{i=1}^k e^{F_i(x)}}$$

$$P(y = 2|x) = \frac{e^{F_2(x)}}{\sum_{i=1}^k e^{F_i(x)}}$$

.....

$$P(y = k|x) = \frac{e^{F_k(x)}}{\sum_{i=1}^k e^{F_i(x)}}$$

其中 $F_1 \dots F_k$ 是 k 个不同的tree ensemble。每一轮的训练实际上是训练了 k 棵树去拟合softmax的每一个分支模型的负梯度。softmax模型的单样本损失函数为：

$$loss = - \sum_{i=1}^k y_i \log P(y_i|x) = - \sum_{i=1}^k y_i \log \frac{e^{F_i(x)}}{\sum_{j=1}^k e^{F_j(x)}}$$

这里的 y_i ($i = 1 \dots k$) 是样本label在 k 个类别上作one-hot编码之后的取值，只有一维为1，其余都是0。由以上表达式不难推导：

$$-\frac{\partial loss}{\partial F_q} = y_q - \frac{e^{F_q(x)}}{\sum_{j=1}^k e^{F_j(x)}} = y_q - \hat{y}_q$$

可见，这 k 棵树同样是拟合了样本的真实标签与预测概率之差，与二分类的过程非常类似。

GBDT多分类流程举例

第一步：训练的时候，是针对样本 x 每个可能的类都训练一个分类回归树。如目前的训练集共有三类，即 $K = 3$ ，样本 x 属于第二类，那么针对样本 x 的分类结果，我们可以用一个三维向量 $[0, 1, 0]$ 来表示，0表示不属于该类，1表示属于该类，由于样本已经属于第二类了，所以第二类对应的向量维度为1，其他位置为0。

针对样本有三类的情况，我们实质上是在每轮的训练的时候是同时训练三颗树。第一颗树针对样本x的第一类，输入是(x,0)，第二颗树针对样本x的第二类，输入是(x,1)，第三颗树针对样本x的第三类，输入是(x,0)。

在对样本x训练后产生三颗树，对x类别的预测值分别是 $f_1(x), f_2(x), f_3(x)$ ，那么在此类训练中，样本x属于第一类，第二类，第三类的概率分别是：

$$P_1(x) = \exp(f_1(x)) / \sum_{k=1}^3 \exp(f_k(x))$$

$$P_2(x) = \exp(f_2(x)) / \sum_{k=1}^3 \exp(f_k(x))$$

$$P_3(x) = \exp(f_3(x)) / \sum_{k=1}^3 \exp(f_k(x))$$

然后可以求出针对第一类，第二类，第三类的残差分别是：

$$y_{11} = 0 - P_1(x)$$

$$y_{22} = 1 - P_2(x)$$

$$y_{33} = 0 - P_3(x)$$

然后开始第二轮训练，针对第一类输入为(x,y11)，针对第二类输入为(x,y22)，针对第三类输入为(x,y33)，继续训练出三颗树。一直迭代M轮，每轮构建三棵树

当训练完毕以后，新来一个样本x1，我们需要预测该样本的类别的时候，便产生三个值 $F_1(x), F_2(x), F_3(x)$ ，则样本属于某个类别c的概率为：

$$P_c(x) = \exp(f_c(x)) / \sum_{k=1}^3 \exp(f_k(x))$$

叶子节点分值计算

对于生成的决策树，计算各个叶子节点的最佳残差拟合值为：

$$c_{m,j} = \arg \min_c \sum_{x_i \in R_{m,j}} L(y_i, F_{m-1}(x_i) + c)$$

我们一般使用近似值代替：

$$c_{m,j} = \frac{\sum_{x_i \in R_{m,j}} r_{m,i}}{\sum_{x_i \in R_{m,j}} (y_i - r_{m,i})(1 - y_i + r_{m,i})}$$

补充近似值代替过程:

假设仅有一个样本:

$$L(y_i, F(x)) = -(y_i \ln \frac{1}{1 + e^{-F(x)}} + (1 - y_i) \ln(1 - \frac{1}{1 + e^{-F(x)}}))$$

令 $P_i = \frac{1}{1 + e^{-F(x)}}$, 则 $\frac{\partial P_i}{\partial F(x)} = P_i(1 - P_i)$

求一阶导:

$$\begin{aligned} \frac{\partial L(y_i, F(x))}{\partial F(x)} &= \frac{\partial L(y_i, F(x))}{\partial P_i} \cdot \frac{\partial P_i}{\partial F(x)} \\ &= -(\frac{y_i}{P_i} - \frac{1 - y_i}{1 - P_i}) \cdot (P_i \cdot (1 - P_i)) \\ &= P_i - y_i \end{aligned}$$

求二阶导:

$$\begin{aligned} \frac{\partial^2 L(y_i, F(x))}{\partial F(x)^2} &= (P_i - y_i)' \\ &= P_i(1 - P_i) \end{aligned}$$

对于 $L(y_i, F(x) + c)$ 的泰勒二阶展开式:

$$L(y_i, F(x) + c) = L(y_i, F(x)) + \frac{\partial L(y_i, F(x))}{\partial F(x)} \cdot c + \frac{1}{2} \frac{\partial^2 L(y_i, F(x))}{\partial F(x)^2} c^2$$

$L(y_i, F(x) + c)$ 取极值时, 上述二阶表达式中的 c 为:

$$\begin{aligned} c &= -\frac{b}{2a} = -\frac{\frac{\partial L(y_i, F(x))}{\partial F(x)}}{2(\frac{1}{2} \frac{\partial^2 L(y_i, F(x))}{\partial F(x)^2})} \\ &= -\frac{\frac{\partial L(y_i, F(x))}{\partial F(x)}}{\frac{\partial^2 L(y_i, F(x))}{\partial F(x)^2}} \xrightarrow{\text{一阶、二阶导代入}} \frac{y_i - P_i}{P_i(1 - P_i)} \\ &\xrightarrow{r_i = y_i - P_i} \frac{r_i}{(y_i - r_i)(1 - y_i + r_i)} \end{aligned}$$

GBDT二分类每轮迭代, 换一种数学符号表达方式:

$$\begin{aligned} \tilde{y}_i &= -\left[\frac{\partial L(y_i, F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)} \right]_{F(x)=F_{m-1}(x)} = y_i - \frac{1}{1 + e^{(-F_{m-1}(x_i))}} \\ \{R_{jm}\}_1^J &= J - \text{terminal node tree}(\{\tilde{y}_i, x_i\}_1^N) \\ \gamma_{jm} &= \frac{\sum_{x_i \in R_{jm}} \tilde{y}_i}{\sum_{x_i \in R_{jm}} (y_i - \tilde{y}_i) * (1 - y_i + \tilde{y}_i)} \\ F_m(x) &= F_{m-1}(x) + \sum_{j=1}^J \gamma_{jm} I(x \in R_{jm}) \end{aligned}$$

GBDT多分类每轮迭代:

$p_k(\mathbf{x}) = \exp(F_k(\mathbf{x})) / \sum_{l=1}^K \exp(F_l(\mathbf{x})), \quad k = 1, K$
 For $k = 1$ to K do:
 $\tilde{y}_{ik} = y_{ik} - p_k(\mathbf{x}_i), \quad i = 1, N$
 $\{R_{jkm}\}_{j=1}^J = J\text{-terminal node tree}(\{\tilde{y}_{ik}, \mathbf{x}_i\}_1^N)$
 $\gamma_{jkm} = \frac{K-1}{K} \frac{\sum_{\mathbf{x}_i \in R_{jkm}} \tilde{y}_{ik}}{\sum_{\mathbf{x}_i \in R_{jkm}} |\tilde{y}_{ik}| (1 - |\tilde{y}_{ik}|)}, \quad j = 1, J$
 $F_{km}(\mathbf{x}) = F_{k,m-1}(\mathbf{x}) + \sum_{j=1}^J \gamma_{jkm} 1(\mathbf{x} \in R_{jkm})$
 endFor

GBDT 二分类举例说明

x_i	1	2	3	4	5	6	7	8	9	10
y_i	0	0	0	1	1	0	0	0	1	1

参数配置：

1. 以logloss为损失函数
2. 以MSE为分裂准则
3. 树的深度为1
4. 学习率为0.1

第一步，初始化。

$$F_0(x) = \log \left(\frac{\sum_{i=1}^N y_i}{\sum_{i=1}^N (1 - y_i)} \right) = \log \left(\frac{4}{6} \right) = -0.4054$$

拟合第一颗树(m=1)

计算负梯度值：

$$\tilde{y}_i = - \left[\frac{\partial L(y_i, F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)} \right]_{F(x)=F_{m-1}(x)} = y_i - \frac{1}{1 + e^{(-F_{m-1}(x_i))}} = y_i - \frac{1}{1 + e^{(-F_0(x_i))}}$$

比如计算第一个样本(i=1)有：

$$\tilde{y}_1 = 0 - \frac{1}{1 + e^{(0.4054)}} = -0.400$$

同样地，其他计算后如下表：

x_i	1	2	3	4	5	6	7	8	9	10
$\tilde{y}_i$	-0.4	-0.4	-0.4	0.6	0.6	-0.4	-0.4	-0.4	0.6	0.6

接着，我们需要以 $\tilde{y}_i$ 为目标，拟合一颗树。拟合树的过程这里就不再赘述了。

拟合完后结果如下：



可以得出建好树之后叶子节点的区域：

R11为 $x_i \leq 8$ ，R21为 $x_i > 8$

下面计算可以叶子节点的值 γ_{jm}

由公式：
$$\gamma_{jm} = \frac{\sum_{x_i \in R_{jm}} \tilde{y}_i}{\sum_{x_i \in R_{jm}} (y_i - \tilde{y}_i) * (1 - y_i + \tilde{y}_i)}$$

对于区域R11有如下：

$$\begin{aligned} \sum_{x_i \in R_{11}} \tilde{y}_i &= (\tilde{y}_1 + \tilde{y}_2 + \tilde{y}_3 + \tilde{y}_4 + \tilde{y}_5 + \tilde{y}_6 + \tilde{y}_7 + \tilde{y}_8) = -1.2 \\ \sum_{x_i \in R_{11}} (y_i - \tilde{y}_i) * (1 - y_i + \tilde{y}_i) &= (y_1 - \tilde{y}_1) * (1 - y_1 + \tilde{y}_1) + (y_2 - \tilde{y}_2) * (1 - y_2 + \tilde{y}_2) \\ &+ (y_3 - \tilde{y}_3) * (1 - y_3 + \tilde{y}_3) + (y_4 - \tilde{y}_4) * (1 - y_4 + \tilde{y}_4) + (y_5 - \tilde{y}_5) * (1 - y_5 + \tilde{y}_5) \\ &+ (y_6 - \tilde{y}_6) * (1 - y_6 + \tilde{y}_6) + (y_7 - \tilde{y}_7) * (1 - y_7 + \tilde{y}_7) + (y_8 - \tilde{y}_8) * (1 - y_8 + \tilde{y}_8) = 1.92 \end{aligned}$$

对于区域R21有如下：

$$\begin{aligned} \sum_{x_i \in R_{21}} \tilde{y}_i &= (\tilde{y}_9 + \tilde{y}_{10}) = 1.2 \\ \sum_{x_i \in R_{21}} (y_i - \tilde{y}_i) * (1 - y_i + \tilde{y}_i) &= (y_9 - \tilde{y}_9) * (1 - y_9 + \tilde{y}_9) + (y_{10} - \tilde{y}_{10}) * (1 - y_{10} + \tilde{y}_{10}) \\ &= 0.48 \end{aligned}$$

故最后可以得到两个叶子节点的值：

$$\gamma_{11} = \frac{-1.2}{1.92} = -0.625, \gamma_{21} = \frac{1.2}{0.480} = 2.5$$

最后通过公式更新

$$F_m(x) = F_{m-1}(x) + \sum_{j=1}^J \gamma_{jm} I(x \in R_{jm})$$

需要注意的是，这里同样也用shrinkage，即乘一个学习率 η ，具体表现为：

$$F_m(x) = F_{m-1}(x) + \eta * \sum_{j=1}^J \gamma_{jm} I(x \in R_{jm})$$

其他计算完毕后如下表供参考：

1	2	3	4	5	6	7	8	9	10
-0.46796511	-0.46796511	-0.46796511	-0.46796511	-0.46796511	-0.46796511	-0.46796511	-0.46796511	-0.15546511	-0.15546511

至此，第一颗树已经训练完成。可以再次看到其训练过程和回归基本没有区别。

下面简单提一下拟合第二颗树(m=2)

计算负梯度值：

比如对于 x_1 有：

$$\Rightarrow \tilde{y}_1 = y_1 - \frac{1}{1 + e^{(-F_1(x_1))}} = 0 - 0.38509 = -0.38509$$

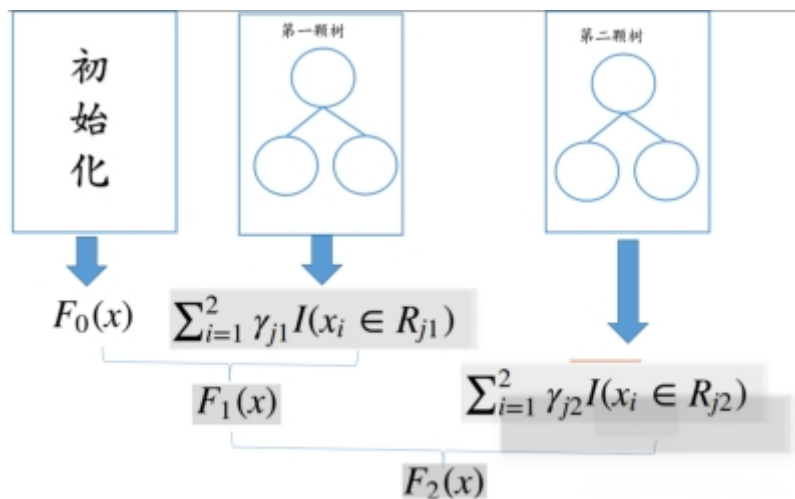
其他同理，可得下表：

x_i	1	2	3	4	5	6	7	8	9	10
$\tilde{y}_i$	-0.38509799	-0.38509799	-0.38509799	0.61490201	0.61490201	-0.38509799	-0.38509799	-0.38509799	0.53878818	0.53878818

之后也是以新的 $\tilde{y}_i$ 为目标拟合一颗回归树后计算叶子节点的区间和叶子节点的值。

关于预测

当只有2颗树的时候，其预测过程也是和下面这个图一样



相比于回归任务，分类任务需把要最后累加的结果 $F_m(x)$ 转成概率。（其实 $F_m(x)$ 可以理解成一个得分）。具体来说：

对于采用logloss作为损失函数的情况下，

$$p_i = \frac{1}{1 + e^{(-F_m(x_i))}}$$

当然这里的 p_i 指的是正样本的概率。

这里再详细一点，比如对于上面例子，当我们拟合完第二颗树后，计算 $F_2(x)$ 可有以下表：

1	2	3	4	5	6	7	8	9	10
-0.52501722	-0.52501722	-0.52501722	-0.52501722	-0.52501722	-0.52501722	-0.52501722	-0.52501722	0.06135501	0.06135501

此时计算相应的概率值有：

$F_2(x)$ 可有以下表：

x_i	1	2	3	4	5	6	7	8	9	10
p_i	0.37167979	0.37167979	0.37167979	0.37167979	0.37167979	0.37167979	0.37167979	0.37167979	0.51533394	0.51533394

(表中的概率为正样本的概率，即 $y_i=1$ 的概率)

GBDT 多分类举例说明

x_i	6	12	14	18	20	65	31	40	1	2	100	101	65	54
y_i	0	0	0	0	0	1	1	1	1	1	2	2	2	2

由于我们需要转化3个二分类的问题，所以需要先做一步one-hot：

x_i	6	12	14	18	20	65	31	40	1	2	100	101	65	54
y_i	0	0	0	0	0	1	1	1	1	1	2	2	2	2
$y_{i,1}$	1	1	1	1	1	0	0	0	0	0	0	0	0	0
$y_{i,2}$	0	0	0	0	0	1	1	1	1	1	0	0	0	0
$y_{i,3}$	0	0	0	0	0	0	0	0	0	0	1	1	1	1

为了方便说明，作以下设置：

1. 树的深度为1
2. 学习率1

首先进行初始化 $F_{k0}(x_i)=0$ ，对所有的样本。

注意：在Friedman论文里全部初始化为0，但在sklearn里是初始化先验概率（就是各类别的占比）

对第一个类别（ $y_i=0$ ）拟合第一颗树($m=1$)。

x_i	6	12	14	18	20	65	31	40	1	2	100	101	65	54
$y_{i,1}$	1	1	1	1	1	0	0	0	0	0	0	0	0	0

$$p_{k,m}(x) = \frac{e^{F_{k,m}(x)}}{\sum_{l=1}^K e^{F_{l,m}(x)}}$$

x_i	6	12	14	18	20	65	31	40	1	2	100	101	65	54
$p_{1,0}$	0.3333	0.3333	0.3333	0.3333	0.3333	0.3333	0.3333	0.3333	0.3333	0.3333	0.3333	0.3333	0.3333	0.3333

下面计算负梯度值，以 x_1 为例($k=1, i=1$):

$$\begin{aligned}\tilde{y}_{ik} &= y_{i,k} - p_{k,m-1} \\ \Rightarrow \tilde{y}_{11} &= y_{1,1} - p_{1,0} = 1 - 0.3333 = 0.6667\end{aligned}$$

同样地，计算其他样本可以有下表：

x_i	6	12	14	18	20	65	31	40	1	2	100	101	65	54
$\tilde{y}_{11}$	0.6667	0.6667	0.6667	0.6667	0.6667	-0.3333	-0.3333	-0.3333	-0.3333	-0.3333	-0.3333	-0.3333	-0.3333	-0.3333

以 y_{11} 拟合一颗回归树，（以31为分裂点）



这里简单补充一下，为什么选择31作为分裂点。

遍历所有特征的取值，将每个特征值依次作为的分裂点，然后计算左子结点与右子结点上的MSE，寻找两者加和最小的一个。

比如，选择1作为分裂点时($x<1$)。左子结点上的集合的MSE为：

$$MSE_{left} = (-0.3333 - (-0.3333))^2 = 0$$

右子节点上的集合的MSE为：

$$MSE_{right} = (0.6667 - 0.02384)^2 + (0.6667 - 0.02384)^2 + \dots + (-0.3333 - 0.02384)^2 = 3.2142$$

故总的MSE为：

$$MSE = MSE_{left} + MSE_{right} = 0 + 3.2142 = 3.2142$$

比如选择2作为分裂点时($x<2$)。

$$MSE_{left} = 0, MSE_{right} = 3.07692, MSE = 3.07692$$

计算完后可以发现，当选择31做为分裂点时，可以得到最小的MSE， $MSE=1.42857$

分别计算 γ_{jkm} 可得：

$$\gamma_{111} = 1.1428, \gamma_{211} = -0.9999$$

$$F_{km}(x_i) = F_{k,m-1}(x_i) + \eta * \sum_{x_i \in R_{jkm}} \gamma_{jkm} * I(x_i \in R_{jkm})$$

利用上式更新可得下表：

x_i	6	12	14	18	20	65	31	40	1	2	100	101	65	54
$F_{1,1}(x_i)$	1.1428	1.1428	1.1428	1.1428	1.1428	-0.999	-0.999	-0.999	1.1428	1.1428	-0.999	-0.999	-0.999	-0.999

至此第一个类别（类别0）的第一颗树拟合完毕，下面开始拟合第二个类别（类别1）的第一颗树：

对第二个类别（ $y_i=1$ ）拟合第一颗树($m=1$)。

x_i	6	12	14	18	20	65	31	40	1	2	100	101	65	54
$y_{i,2}$	0	0	0	0	0	1	1	1	1	1	0	0	0	0

$$p_{k,m}(x) = \frac{e^{F_{k,m}(x)}}{\sum_{l=1}^K e^{F_{l,m}(x)}}$$

x_i	6	12	14	18	20	65	31	40	1	2	100	101	65	54
$p_{2,0}$	0.3333	0.3333	0.3333	0.3333	0.3333	0.3333	0.3333	0.3333	0.3333	0.3333	0.3333	0.3333	0.3333	0.3333

下面计算负梯度值，以 x_1 为例($k=2, i=1$):

$$\begin{aligned} \tilde{y}_{ik} &= y_{i,k} - p_{k,m-1} \\ \Rightarrow \tilde{y}_{12} &= y_{1,2} - p_{2,0} = 0 - 0.3333 = -0.3333 \end{aligned}$$

同样地，计算其他样本可以有下表：

x_i	6	12	14	18	20	65	31	40	1	2	100	101	65	54
$\tilde{y}_{i2}$	-0.3333	-0.3333	-0.3333	-0.3333	-0.3333	0.6667	0.6667	0.6667	0.6667	0.6667	-0.3333	-0.3333	-0.3333	-0.3333

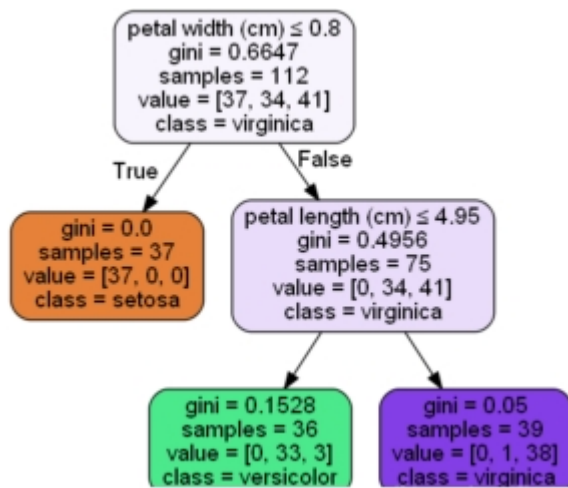
以 y_{i2} 拟合一颗回归树，（以6为分裂点），可计算得到叶子节点：

$$\gamma_{121} = 2, \gamma_{221} = -0.2499$$

然后再拟合第三个类别（类别2）的第一颗树，过程也是重复上述步骤，所以这里就不再重复了。在拟合完所有类别的第一颗树后就开始拟合第二颗树。反复进行，直到训练了M轮。

特征选择

- ✓ 特征j的全局重要度通过特征j在单颗树中的重要度的平均值来衡量
- ✓ 特征j在单颗树中的重要度，是计算特征j在单颗树中带来的收益之和



petal width (cm)就是根节点:

feature importance=

$$(112 \times 0.6647 - 75 \times 0.4956 - 37 \times 0) / 112 = 0.5564007189$$

petal length (cm)的

feature importance=

$$(75 \times 0.4956 - 39 \times 0.05 - 36 \times 0.1528) / 112 = 0.4435992811$$

Friedman在GBM[1]的论文中提出的方法:

特征 x_j 在整个模型中的重要程度为:

$$\hat{J}_j^2 = \frac{1}{M} \sum_{m=1}^M \hat{J}_j^2(T_m)$$

其中, M 是模型中树的数量。特征 x_j 在单独一个树上的特征重要度为:

$$\hat{J}_j^2(T) = \sum_{t=1}^{L-1} \hat{I}_t^2 1(v_t = j)$$

其中, $L - 1$ 是树中非叶子节点数量, v_t 表示在内部节点 t 进行分裂时选择的特征, $\hat{I}_t^2$ 是内部节点 t 分裂后平方损失的减少量。

GBDT用于特征组合降维

